GAMMA 60
Data Processing System

Compagnie des Machines Bull

1961

Digitalized in 2024 from original:

Archives nationales du monde du travail, Roubaix, France
2012 007 *Bull, informatique* / 3198

Project JuraSTIC

In collaboration with

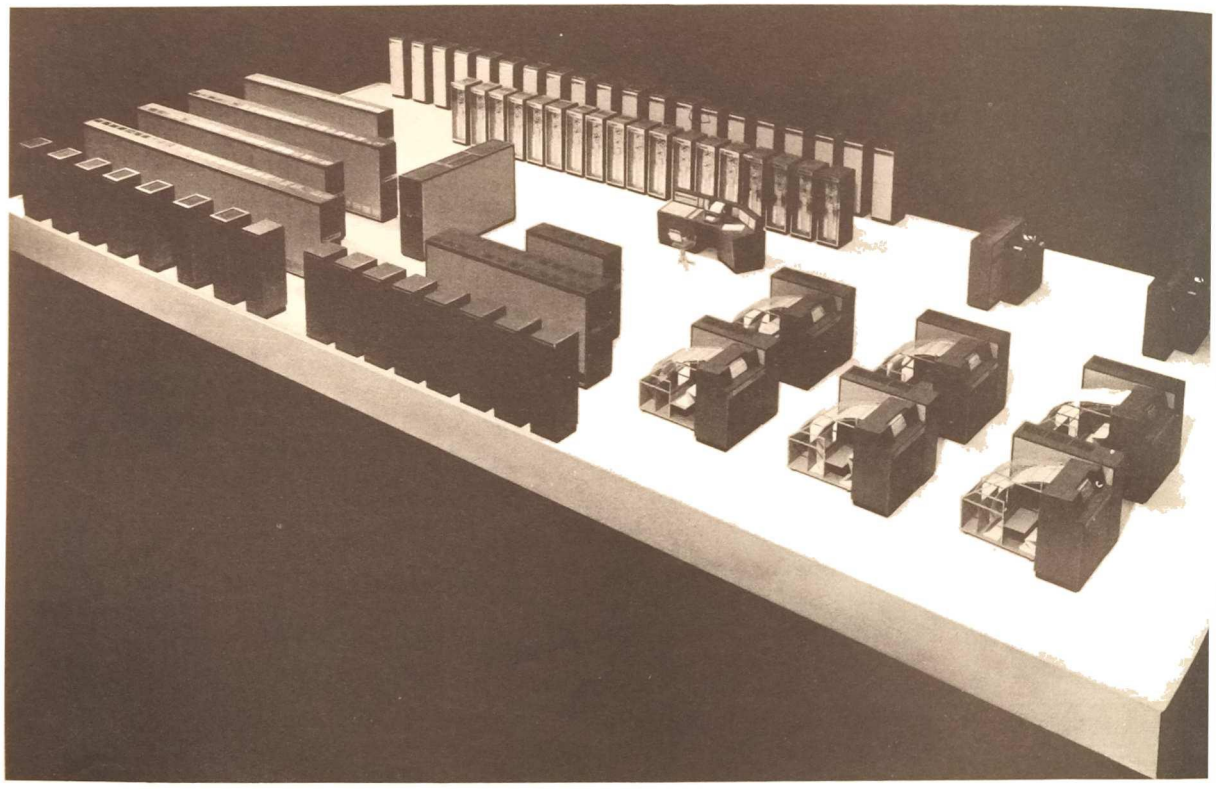
gamma 60

Data processing system

1961

gamma 60

- DATA IN THE GAMMA 60
- GENERAL DESIGN
- ELEMENTARY INSTRUCTIONS
- DATA PROCESSING UNITS
- INPUT OUTPUT UNITS
- DATA STORAGE UNITS



Introduction

Calculating by mechanical means is not new. Pascal's calculating machine is more than three hundred years old. On the other hand, not until some fifteen years ago did men begin to realize the immense possibilities electronics had to offer in the field of automatic calculating. Nevertheless, very considerable ground has already been covered between the designing of the first electronic calculators and the large-scale computer systems of to-day.

At first the aim was speed and more speed. The resources of electronics were applied to the construction of machines directed primarily towards numerical calculating. They were the first electronic calculators. Their performance represented such an advance on those of electromechanical machines that it was often impossible to resist speaking of them in superlatives of highest praise, sometimes even a little rashly, as in referring to them as "Electronic Brains". Although, taken literally, such an expression would be an exaggeration, the fact remains that, in the same way as the human brain can do many things besides calculate, so electronic machines, based on the same principles as the calculators, can perform work going far beyond the function of calculating.

Indeed, it has rapidly become evident that numerical calculation consists basically of a very limited number of elementary logical operations that recur whenever data is to be selected, compared or distributed, etc. all the operations that go to make up what is known as "data processing", a much wider concept than that of calculation. The next step was obviously to build up the electronic circuits that perform elementary logical operations into machines for processing information in general. The present-day "Large-scale Electronic Data-Processing Systems" thus came into being.

The idea of a machine for processing information is not in itself new; the punched card tabulator is the earliest example of it in practice. The tabulator was designed to produce statistical, administrative or accounting statements, which require not only calculations, but also and especially, distribution and selection operations, and in general, all the operations involved in editing statements in the form required.

However, the resources of electronics could not be applied to the vast field of data processing purely and simply by tacking them on to machines with logical structures

closely akin to those of the old electromechanical machines. Alongside the technological effort, a special effort was necessary in order to evolve a logical machine design allowing the best possible use of electronic technology.

The Gamma 60, introduced by the Compagnie des Machines Bull, is the combined result of these two efforts, which makes it a perfectly balanced machine representing the peak in development of large-scale electronic systems.

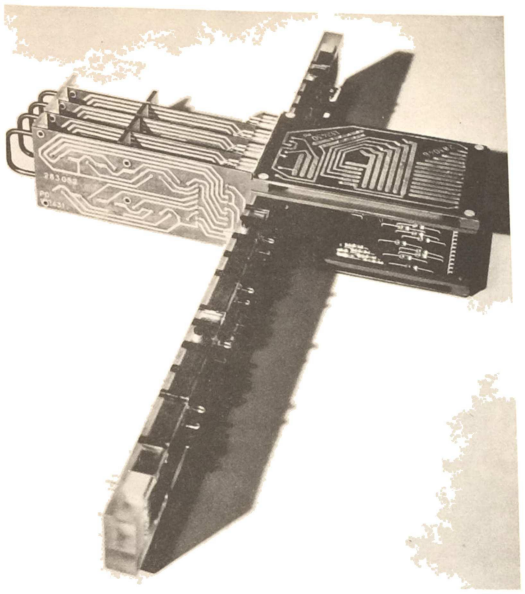
It makes use of the latest technological advances proved by industry to be the most reliable. Apart from some power tubes in the central unit, the electronic circuitry is made entirely of semi-conductors, diodes and transistors. Magnetic-core stores are widely used and printed circuits systematically employed.

In order to standardize manufacture, facilitate mounting, ensure the safety of the components and easy access in case of repairs, electronic "cabinets" as well as wiring "cabinets" have been produced. These contain numerous plug-in printed-circuit plates fitted with the active elements, diodes and transistors. The Gamma 60 is of course a magnetic-tape machine and magnetic drums can also be used as handy auxiliary stores.

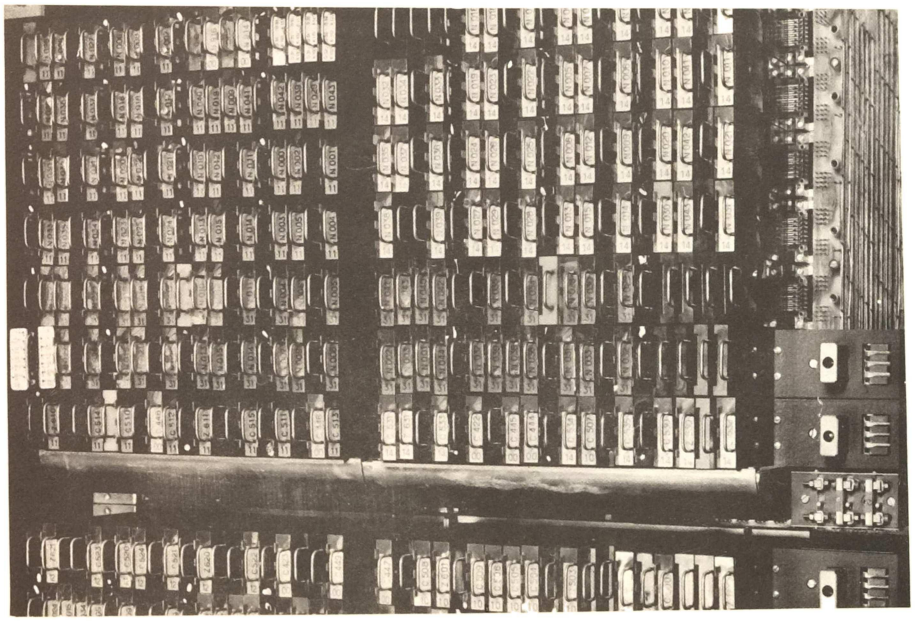
As a matter of fact, all these technological media are the components of an entirely new logical structure which enables the machine to deal simultaneously with several jobs, each being processed in the same time as if processed separately. Simultaneous operation is in fact the governing principle of the Gamma 60, as will be seen from the following pages.

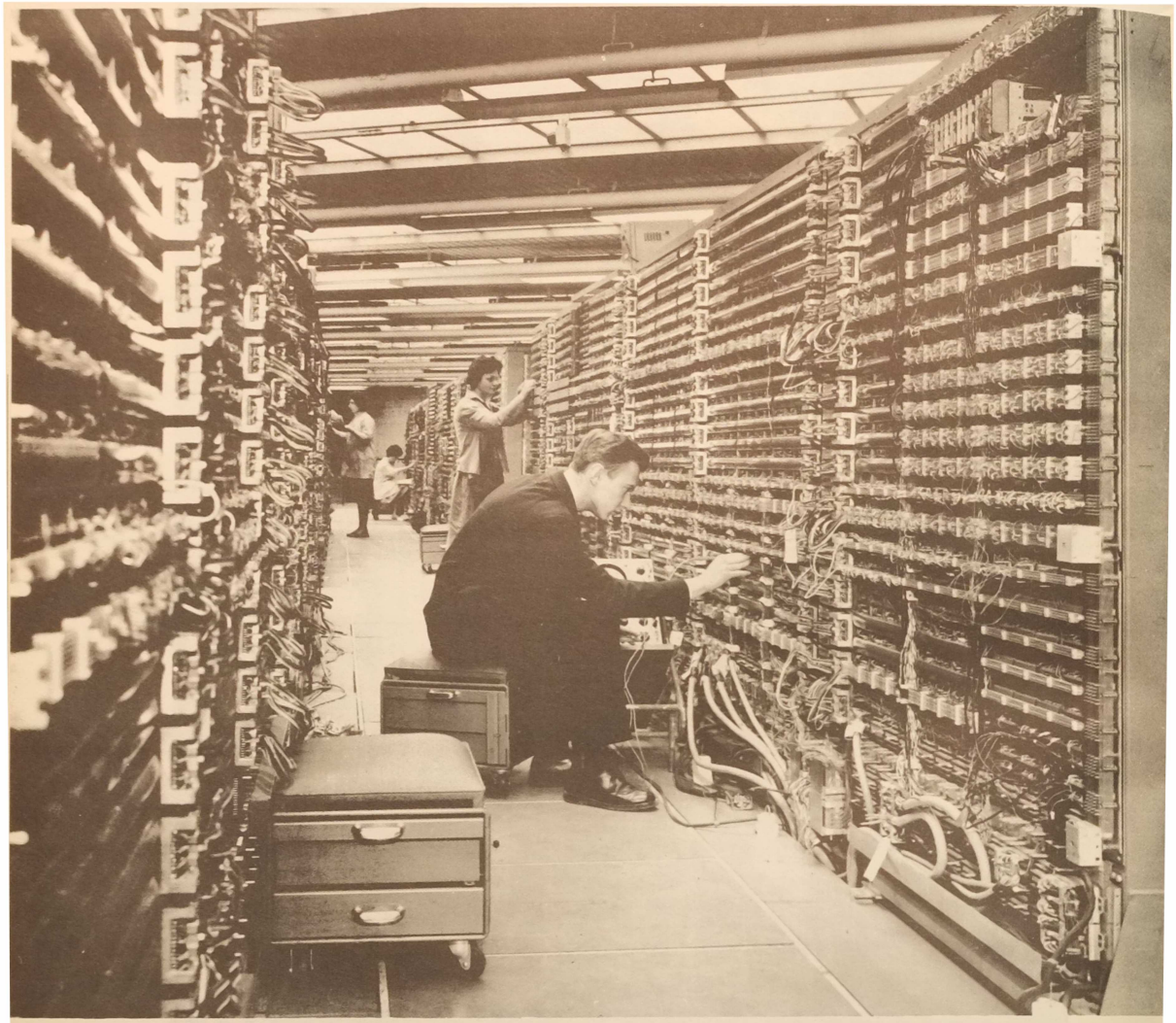
The main object of this brochure is to describe the design of the Gamma 60, its internal language, its various parts and their functions. Another brochure will deal with programming, and more generally, with study methods required for problems to be solved by the Gamma 60. In connection with the internal language, we will refer to machine instructions; these however, should be looked upon as being merely the material used for elaboration of programming methods. Although it is useful to have some knowledge of them, they do not in themselves, constitute a programming technique, as will be shown in the second brochure.

A "section" is a set of four transistorized amplifiers connected with logical circuits. The structure of the plug-in elements is clearly visible. A Gamma 60 has approximately 3.600 sections.



The sections are grouped in "shelves" which form the "cabinets" of the Gamma 60.





Final assembly check of the central unit

1

Data in the Gamma 60

I - Data and machines

The term "data processing" denotes any process by which given data is worked on in order to produce other data incorporating preceding information.

As long as the processing goes on, a machine must be able to memorize both the data and the way in which it is being dealt with, in other words, the "programme". With electromechanical machines such as the tabulator, the programme is "posted" up on a "plugboard". Large-scale electronic systems like the Gamma 60, are known as "stored-programme" machines. Their programmes, which describe the work to be done as a succession of elementary operations, consist of series of "instructions" coded in a language which only the machines understand. These instructions, together with the data to be processed, have to be stored in "memories" and to remain available throughout the processing time.

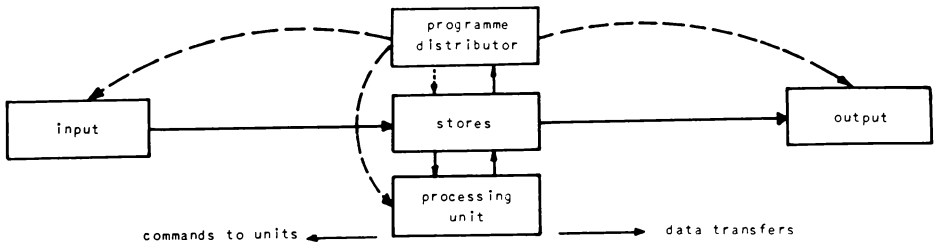
The complexity of a problem stems either from the large volume of data to be processed, or the elaborate operations to be carried out, or from both. As a result, the more complicated a problem is, the larger is the amount of information to be stored, whether data to be processed or programme instructions. Hence the importance of the "memory" function in these machines. The larger the storage capacity, the better is the machine fitted to produce in a single processing phase, results which respond to numerous parameters. Such machines are often said to perform integrated processing : the Gamma 60 heads this class. Conversely, machines with lower storage capacity make it necessary to break the processing down into successive phases so as to build up, step by step, the complicated results required. This is what happens with punched card electromechanical machines.

The entire layout of a data processing machine hinges on the storage units.

- Input units supply data to be processed to the storage units. Input medium is punched card or paper tape (it is also quite likely in the near future to be magnetically printed paper and even plain printed paper).
- Processing units (calculators, comparators, etc.) work out the results.
- Output units, then issue the results, either on punched cards or paper tape or, more often, print them out.

The operation of all these units is supervised and co-ordinated by programme units whose task it is to interpret the instructions stored along with the data. Like the data to be processed, the programme is supplied by means of the input units.

Broadly speaking, the layout of any data processing machine is therefore the following :



II - The store function

The above statements emphasizing the importance of the store function do not, however, imply that at any time in the course of processing one needs to be able to pick out instantaneously any of the stored information. The data must all be stored in one way or another, for the processing units to be able to use it as needed, but the technological media used for this storing process, i.e. the stores themselves, may very well vary in type so that the cost of meeting the demands made on them is kept to the minimum.

To illustrate this : while a payroll job is in progress, the calculation-programme instructions and the wage data for the particular operative must be contained in a store which will at any moment allow immediate access to any item of this data without in the least delaying the operation of the processing units. However, the sum total of data on the wages of all the operatives can be recorded in a store so designed that data is accessible only in the sequence in which it will be required. Once the wages of one operative have been worked out, the data enabling the next operative's wages to be calculated is transferred from this second type of store - the magnetic tape type, for instance - into the first type - a high speed magnetic-core store. Magnetic tape is, of course, far less expensive than the core store.

Therefore, although it is true that any device able to record, hold and redeliver data is called a store, it is equally true that these stores can be classified according to various criteria which are used to determine which type of memory is best suited for a given application.

An initial distinction is made between *direct-access stores* and *sequential-access stores*. A store is said to be of the direct-access type when a piece of information is immediately accessible, wherever it happens to be in the store, without it being necessary to run through other data recorded in the same store; these stores are the magnetic-core or to some extent the magnetic-drum type. In contrast, magnetic tape is the standard for sequential-access stores. To gain access to a given item of data, the section of tape between the reading head and the data required must first be unwound under the head. For these stores to be used efficiently, data should therefore be arranged in them in the order in which it is to be used.

A second distinction is made between *addressable* stores and *non-addressable* stores. A store can be divided up into a certain number of store cells, each of which is numbered; these numbers are known as *addresses* and their function is to differentiate and label each cell. For a store to be of the direct-access type, it must be addressable. However, an addressable store is not necessarily of the direct-access type; it is possible to design an addressable magnetic tape system but, owing to its actual physical nature, access to this type of store can only be sequential.

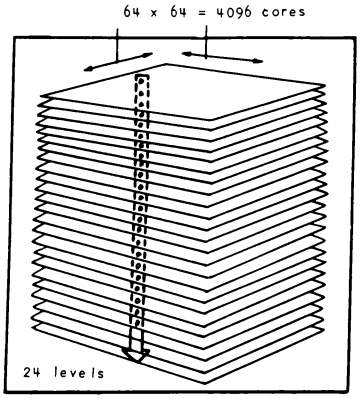
A final distinction is made between *internal* stores and *external* stores. The former (magnetic-core stores and magnetic drums) are an integral part of the machine. They are intended to contain data which is to be worked on constantly, and their storage capacity is one of the factors, but not the only one, which determines whether the machine is equipped to carry out integrated processing. For the very reason that these stores are an integral part of the machine, they cannot be used for the inter-operational storage, from one month to the next, for instance, of data in bulk such as the sum total of permanent data concerning each operative in a factory and used for payroll operations. This sort of data is entered in files and recorded by external stores (magnetic tape, punched cards or punched paper tape) which are fed into the input units of the machine when they are required.

The stores are further characterized by two important measurements : *capacity* and *access time*. The capacity of a store is the measurement of the volume of data that it can hold. The storage capacity of internal stores is relatively low compared with that of external stores, which is virtually unlimited. The notion of access time has no meaning unless the store concerned is the direct-access type. The term can be defined as the time that elapses between the moment a given item of data at a given address in the store is requested and the moment when this item is actually obtained. It is obvious that this notion is devoid of meaning where sequential-access stores, like magnetic tape, are concerned, since this time lag may vary from nothing to several minutes (the time it takes to read an entire tape). Stores which, due to their physical make-up, have very short access times, usually also have a low storage capacity owing to their high cost.

III - The stores in the Gamma 60

1. The central store

This store is so called because of the leading part it plays in the design of the Gamma 60. As will be seen later, it is a genuine meeting-place where all the data used for the current operation, whether programme instructions or data to be processed, is to be found at any one moment. Also, it is called a "high-speed store" as it is the quickest-access store in the Gamma 60.

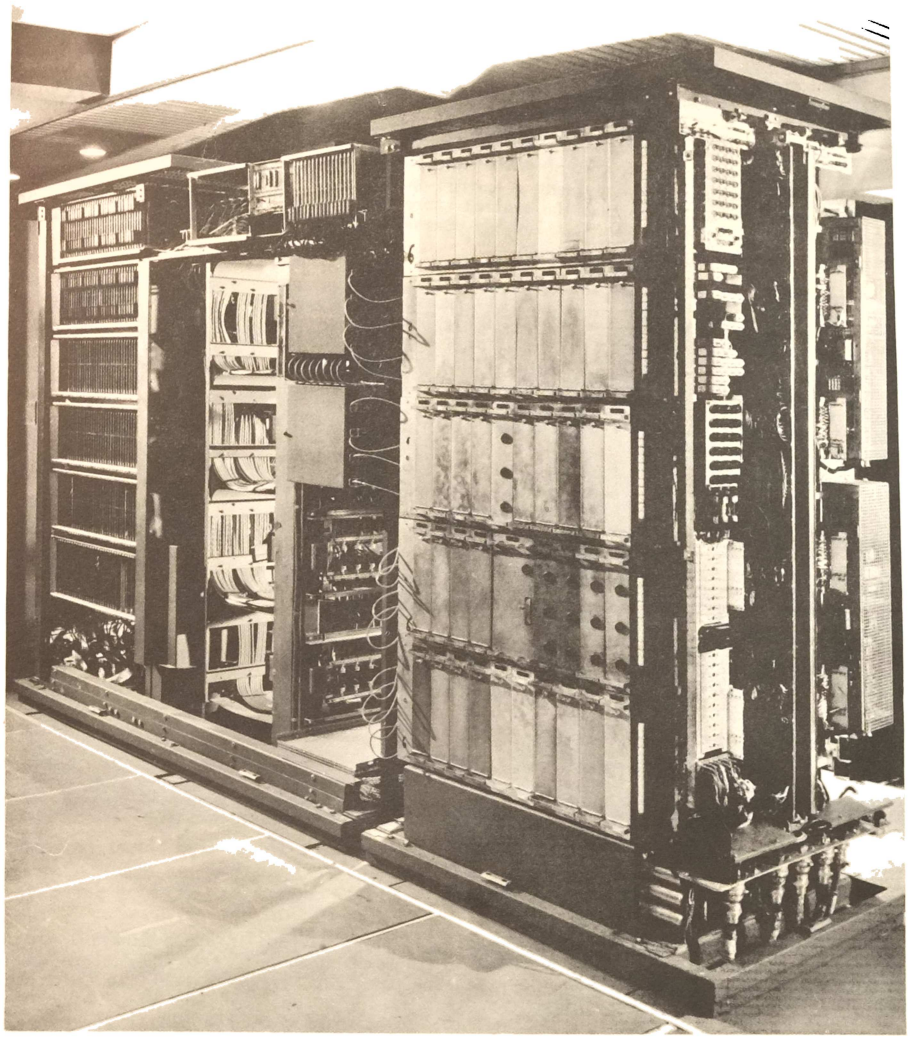


Physically speaking, it is a static store made up of a very great number of ferrite magnetic cores set at the intersections in a network of conductor wires whose function it is to record or read data contained in the cores. Each core can have two stable states of magnetization, corresponding to the symbols 0 and 1, and can therefore contain an elementary unit of data known as a "bit".

The ferrite cores are arranged in blocks of 24 superposed levels of 4096 cores (64 × 64). A block may also be thought of as being made up of 4,096 columns of 24 superposed cores. Taken as a whole, the 24 bits contained in the 24 cores form the data quantum used in the transfers between the central memory and the various units in the Gamma 60. This data quantum is known as a "catena".

A central store is made up of from 2 to 8 of these blocks. Since data coding for the Gamma 60 is such that a catena can contain either 6 decimal digits or 4 letters, the capacity of the central store is given in the following table as per the number of blocks (only the most usual sizes, i.e. 2, 4 and 8 blocks, are given).

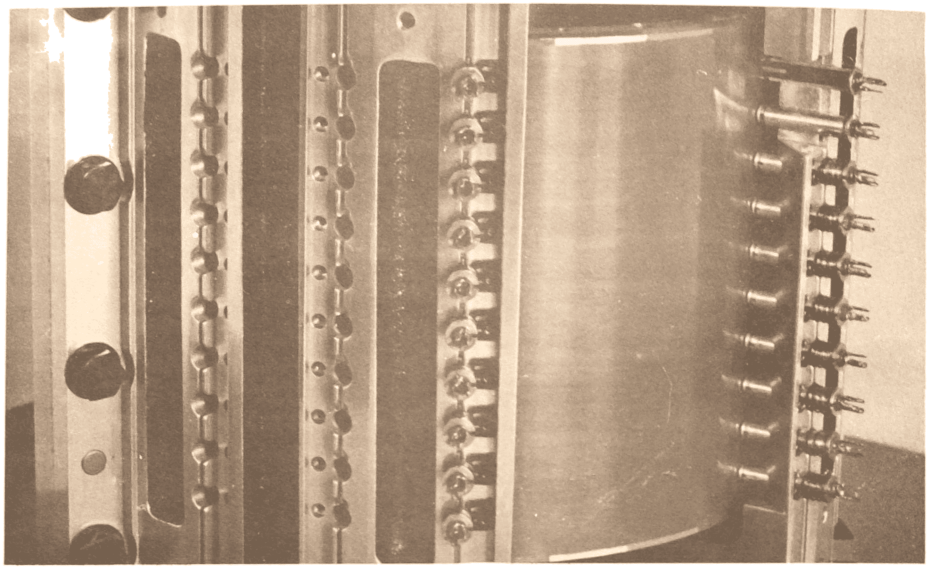
The central store is *addressable by catena*, each column forming a store cell to which an address between 0 and 32,767 is given. These addresses are made commonly available to both programme instructions and data, i.e., address 3,500, for instance, may contain a programme instruction during one phase of the operation and data to be processed during another.



Central store. The size of the central store can be extended from two to eight high speed store blocks. The above photo shows two blocks already installed and space for six additional blocks. To the right may be seen the reading and writing amplifiers and on the left, the circuits for selection of the addresses.

	2 blocks	4 blocks	8 blocks
Binary digits	*196,608*	*393,216*	*786,432*
Decimal digits	*49,152*	*98,304*	*196,608*
Alphanumerical characters	*32,768*	*65,536*	*131,072*
Catenae	*8,192*	*16,384*	*32,768*

At a given moment, the central store can have an input or output of only one catena at a time. The time taken by the input or output operation is 10 microseconds. Another way of expressing this is to say that the central store has a 10-microsecond cycle. This can, moreover, be done at any moment and for any adress. In other words, the *access time* of the central store is 10 microseconds.



A magnetic drum with its read and write heads

2. The magnetic drum

This is a cylinder coated with a magnetizable layer divided into 128 parallel circular tracks, each of which has a read and write head. Read and write operations can be carried out only on one track at a time. The drum revolves on a vertical spindle at 3,000 r.p.m., a single revolution therefore taking 20 ms.

Each track can accommodate 200 catenae; total drum storage capacity therefore amounts to 25,600 catenae, i.e., 153,600 digits or 102,400 letters.

The magnetic drum is an addressable store, each catena on it having an address, from 0 to 25,599.

The access time required to obtain an item of information depends on the position of the latter in relation to the read head at the time that the transfer is commanded; it can vary from 0 to 20 microseconds (the time taken for one revolution).

The average access time is therefore 10 microseconds.

The time taken to read or write one catena on the drum is 100 microseconds. Data transfers from the central store to the drum are variable in length with the proviso that this length should be a complete number of catenae.

3. Magnetic tape

This is the ideal method of storing large amounts of data cheaply. It consists of mylar plastic tape 12.7 mm wide, covered with a layer of magnetic oxyde and is handled by tape units which include :

- devices for unreeling and rereeling the tape,
- a write head,
- a read head which also serves to check the data being written on the tape by rereading it.

The magnetic tape is divided up into eight recording tracks, six of which are for the actual data, the remaining two being for check data (see chap. 2, § V). Each catena thus covers four consecutive columns crosswise on the tracks. Catenae are grouped and recorded in *blocks which vary in length* from 16 to 4,095 catenae to allow for stopping and starting in the course of operation. At the beginning and at the end of each block is a catena which contains the length of the block expressed in bits. Each block is separated from the adjoining block by start/stop spaces 2 cm long.

Recording density inside the blocks is 27.5 catenae to the centimetre. As tape comes in 1,100 m or 730 m spools, and on the assumption that the average length of the blocks is 250 catenae, spool capacity is as shown in the following table :

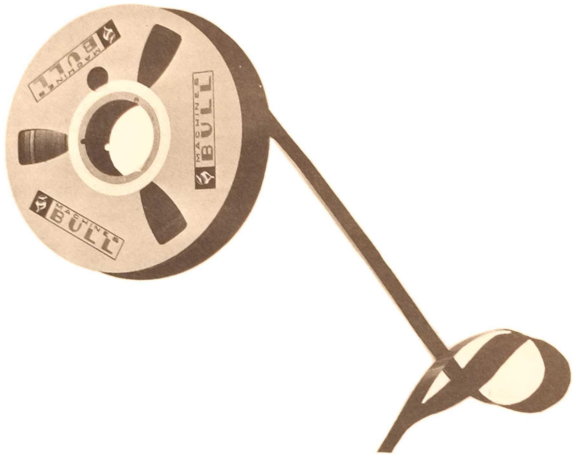
CAPACITY	*730 m*	*1,100 m*
catenae	*1,600,000*	*2,400,000*
letters	*6,400,000*	*9,600,000*
digits	*9,600,000*	*14,400,000*

The linear speed of tape for both reading and writing is 1.90 m/sec. Given the recording density, this corresponds to a working *frequency* of 21 Kc/s per track.

Under the above conditions, the reading time for one catena is 190 *microseconds*, which corresponds to an instantaneous flow of 21,000 alphanumerical characters or 31,500 digits per second. On the continued assumption that blocks consist of an average of 250 catenae, and on the additional assumption that the tape is run continuously, by incorporating the running time for the start/stop spaces, an *average reading time* of 230 microseconds *per catena* is obtained.

A 730 m tape takes 6 minutes 20 seconds to read and a 1,100 m tape, 9 minutes 30 seconds.

Magnetic tape recordings can be made only in one direction, but tape can be read in both directions.



A magnetic tape reel

4. Comparison between the stores

The following table summarizes the main features of the Gamma 60 stores. Punched cards and punched paper tape have been included. Although not exclusive to the Gamma 60, they nonetheless form its external stores as they are the normal media for data inputs. The punched cards used are of the standard, 80-column type each of which can hold 80 digits or alphanumerical characters. They are read on each of the Gamma 60 card-readers at the rate of 300 cards per minute. The punched paper tape is of the standard 5 or 8-track type and can be used with any code, as will be seen later on. The tape is usually in 150 m lengths and holds 10 characters per inch; it is read on paper-tape readers at the rate of 300 characters per second.

In order to obtain comparable figures in each column of the table, the catena has been taken as the unit measurement of data volume and all figures expressed in terms of this unit, and punched cards and paper tape have been considered as holding half digits and half alphanumerical characters, i.e.. an average of 5 characters per catena.

store	storage capacity in catenae	access time	catena reading time
high - speed store (1 block)	4,096	10 µs	10 µs
magnetic drum	25,600	10ms (average)	100 µs
magnetic tape 730 m	1,600,000		230 µs (average)
1,100 m	2,400,000		
punched paper tape (150 m)	11,800		16.6ms
punched card	(per card) 16		12.5ms

The figures for the punched cards assume that all the columns are used, while this is rarely the case. Punched card and punched paper tape data input speeds can therefore be considered as being to all intents and purposes the same.

IV - The internal language of the Gamma 60

As has already been mentioned, data transfers between one unit of the Gamma 60 and the central memory are divided up into catenae of 24 bits each. However, for the data to be processed, it must be coded within these bits in a language comprehensible to the Gamma 60. The various types of data that a catena can contain will therefore now be examined from this viewpoint.

1. Programme instructions

Since the catena is the transfer quantum of the Gamma 60, it is natural that programme instructions should, in machine language, be one catena long, and that, when the programme is to be carried out, instructions should be located in consecutive addresses in the high-speed store. These one catena-long instructions are known as *elementary instructions*. Their code, known as code A, is the only instruction code that the programme circuits are able to analyse and use for the control of the various units.

No further mention will be made of code A, which is pure binary and never used by programmers. They employ symbolic codes only and these will be discussed later.

However, from the structural point of view, it should be noted that elementary instructions are divided into two parts :

- the first part consists of the 9 most significant bits of catena (bits 16 to 24), and indicates the type of instruction by setting the various conditions for carrying out the corresponding operation.

- the second part consists of the 15 least significant bits of catena (bits 1 to 15), and contains a 15-digit binary number which denotes either a binary number or a central store or drum address, or a number of catenae or bits. As the highest high-speed store address is 32,767, this is expressed in pure binary notation by a 15-digit number, all the digits being 1's.

2. Decimal digits

Purely numerical data, which is expressed in decimals in the normal way, retains this character in the Gamma 60. However each decimal digit is coded in binary notation, thus using 4 bits; a catena can therefore hold 6 decimal digits, e.g. :

1001	1000	0110	0000	0010	0011
9	8	6	0	2	3

This numerical code is known as the "binary coded decimal" notation.

3. Alphanumerical characters

The wording may include digits, letters and possibly other characters such as punctuation marks. All these characters have the same data value. For instance, in a name and address such as :

BULL, 94 AVENUE GAMBETTA, PARIS

GAMMA 60 CODE

INTERNAL CODE				00			01			10			11	
				Character	Card Code		Character	Card Code		Character	Card Code		Character	Card Code
0	0	0	0	space			P	8	4	0		0	Δ	9 7 0
0	0	0	1	A	7	11	Q	8	5	1		1	:	9 7 1
0	0	1	0	B	7	0	R	8	6	2		2	'	9 7 2
0	0	1	1	C	7	1	S	9	11	3		3	×	9 7 3
0	1	0	0	D	7	2	T	9	0	4		4	%	9 7 4
0	1	0	1	E	7	3	U	9	1	5		5	<	9 7 5
0	1	1	0	F	7	4	V	9	2	6		6	>	9 7 6
0	1	1	1	G	7	5	W	9	3	7		7	/	9 7
1	0	0	0	H	7	6	X	9	4	8		8	(	9 8 0
1	0	0	1	I	8	12	Y	9	5	9		9	)	9 8 1
1	0	1	0	J	8	11	Z	9	6	*		12	-	9 8 2
1	0	1	1	K	8	0	.		11	,	9	8 11	+	9 8 3
1	1	0	0	L	8	1				◊	9	12	≠	9 8 4
1	1	0	1	M	8	2				-	9	8	Π	9 8 5
1	1	1	0	N	8	3				£	9	7 11	Σ	9 8 6
1	1	1	1	0	7	12				$	9	7 12	□	9 8 12

the data value of "94" is the same as that of "AVENUE" and of the spaces or the commas. In order to represent such data, it is necessary to adopt an alphanumerical code which allows a sufficient quantity of distinct combinations. In the case of the Gamma 60, each alphanumerical character is represented by a combination of 6 bits (64 possibilities) which enables the 60 characters used to be represented :

26 letters
10 digits
23 punctuation marks and special characters
space

A catena can therefore hold 4 alphanumerical characters, e.g.:

000010	010101	001100	001100
B	U	L	L

The table opposite gives this alphanumerical code. The last four bits of the 4 characters to which these four bits are common, are to be found in each line. The first two bits which differentiate each of these characters are to be found at the top of each column.

In order to supply these 60 characters to the machine on punched cards, it was necessary to develop a punched-card code with 3 perforations per column. This code, more powerful than the Bull code for electro-mechanical equipment, is also shown in the table, next to the corresponding characters. It will be noticed that it is the same as the standard Bull code so far as letters and digits are concerned : digits are represented by one perforation on the corresponding line, and letters by a combination of two perforations. The only exceptions are the letters O and I which are distinguished from digits 0 and 1, as in the internal code. Triple-perforation combinations are used only for special signs.

Concerning the internal alphanumerical code for digits, it will be noticed that it is deduced from the decimal binary code by adding 10 in front. Consequently, if each character is interpreted as a 6-digit binary number (from 0 to 63), the letters come before the digits, this being the standard form of alphanumerical classification.

4. Pure binary digits

A catena can hold a positive binary number of 24 digits. The Gamma 30 can carry out the four arithmetic operations with binary numbers of a maximum of 24 digits.

5. Qualitative data

The 24 bits in a catena can be considered as so many tests to which the desired conventional meaning can be applied. The purpose of these tests is that they are used by the programme in order to make it evolve in the desired manner. They can either describe the structure of a set of data (geographical catena), or put into concrete form some qualitative results obtained in the course of one part of the operation and which are to be used later on (qualitative catena : chap.2, § III - 3).

6. Representation of algebraic numbers

Any number on which calculations are to be done consists of a sign and an absolute value. This absolute value can always be written as the product of a value lower than 1, known as a "mantissa", multiplied by a power of 10. There are an infinity of

ways in which this can be expressed, e.g. :

324.5 can be written 0.3245×10^3, or 0.003245×10^5, etc.

0.00482 can be written 0.482×10^{-2}, or 0.00482×10^0, etc.

A number can therefore be characterized by its sign, a power of 10 and its "mantissa".

In order to avoid operations with negative exponents, 40 is automatically added on to them when using the Gamma 60. In addition, the mantissa is represented only by the digits to the right of the 0 and its capacity does not exceed 10 digits. This is known as the *semi-logarithmic representation* of numbers in the Gamma 60.

For this purpose, a number occupies two catenae. In the first are to be found the sign, the exponent and the first four digits of the mantissa ; in the second are the six last digits of the mantissa. The sign and the exponent are in the two tetrads to the left of the first catena ; the first bit represents the sign (0 for + 1 for -); the three remaining bits of the first tetrad represent the first digit of the exponent (from 0 to 7) in binary code, the second tetrad represents the second digit of the exponent (from 0 to 9), and the ten remaining tetrads represent the mantissa in binary coded decimal notation.

For instance : + 0.00482 could be expressed as :



The exponent can always be chosen from amongst the various possible ways of representing a number so that the mantissa is between 0.1 and 1 for the first digit following the exponent to be significant. The Gamma 60 can do this automatically in the course of operations and this enables results of maximum possible relative accuracy to be obtained. This is known as carrying out a floating decimal point notation. e.g. : + 325.42 would be as follows in a floating decimal point notation :

+ 43.3254	200000
1st cat.	2nd cat.

However, in accountancy problems, it is particularly advantageous to operate so that the decimal point is always in the same place for results of the same type. The Gamma 60 is then said to carry out a fixed decimal point rotation. The various possible positions for the decimal point are marked as shown in the above figure. The exponent of a fixed decimal point number is always between 40 (f.d.p. to 10) and 50 (f.d.p. to 0).

For instance : + 4832 is expressed by :

+ 50.0000	004832	(with decimal point in position 0)
1st cat.	2nd cat.	

+ 45.0483	200000	(with decimal point in position 5)
1st cat.	2nd cat.	

2

General design of the Gamma 60

I - The governing principle of the Gamma 60

Like all data processing machines, the Gamma 60 comprises :

- input, output and processing units, which will be called "elements", pending the provision of a more exact definition.
- store and programme units, which form the central unit.

The originality of the structure of the Gamma 60 stems from the following two points :

1. *Each element is specialized* in the performance of certain functions. This is so in all machines for input/output units : a card reader is obviously specialized in the reading of punched cards, a printer in printing, etc. But it is not always so with processing units. In most machines, the calculation, selection and comparison units are regarded as forming a logical part of the central unit and their circuits are closely integrated with those of the programme units. In the Gamma 60, the processing units are "elements", which, in relation to the central unit, allow of the same logical adaptation as the input/output elements. There are 4 processing elements, each specializing in a well-defined type of function :

- an arithmetic calculator for arithmetic and algebraic calculations with decimal numbers expressed in semi-logarithmic form.
- a logical calculator for the calculation of logical sequences and arithmetic calculations with pure binary numbers.
- a general comparator for classification,
- a transcoder for transcoding the external code of cards or punched paper tape into the internal code of the Gamma 60.

2. However, the specialization of each element assumes its full meaning and efficiency only through the *operational autonomy* of these elements. It is the independence of the elements which is the *fundamental characteristic* of the Gamma 60 : the purpose of this chapter is to define its precise scope and limits. It should be noted forthwith that for the user it entails the following chief consequence :

It makes it automatically possible for *several elements to operate simultaneously*. This means that *several programme sequences can be run simultaneously* in the Gamma 60, each of them being, at each moment, taken up by an element and dealt with in the same length of time as if there was only one problem in the machine. These simultaneous sequences can take place whether they are part of the same programme, or part of entirely independent problems. In practice, problems as different as rocket trajectory and payroll computation can be treated at the same time by the Gamma 60. It is also possible to deal at the same time with magnetic tape file up-dating, one or several tape-to-printer conversions and one or several card inputs, even if these operations concern different problems.

II - Brief description of the structure of the Gamma 60

In the Gamma 60 there is no direct connection between elements. Looked at from this angle, they are independent of one another. The unity of the machine is ensured by the connection of all the elements to a central "meeting-place", to which all elements have two-way access through two channels which are common to all the elements.

This "meeting-place" is the *central store* (already described in the previous chapter) which is the main component of the central unit. It contains permanently, the instructions for the working elements and the data they are to process. Its task is also to receive the results produced by the elements working on their own. When the result produced by one element is to be used as data by another one, it is not possible to transfer it directly to this second element because the two are not connected up directly. The first element has to send the result to the central store, and the second has to go and fetch it from there.

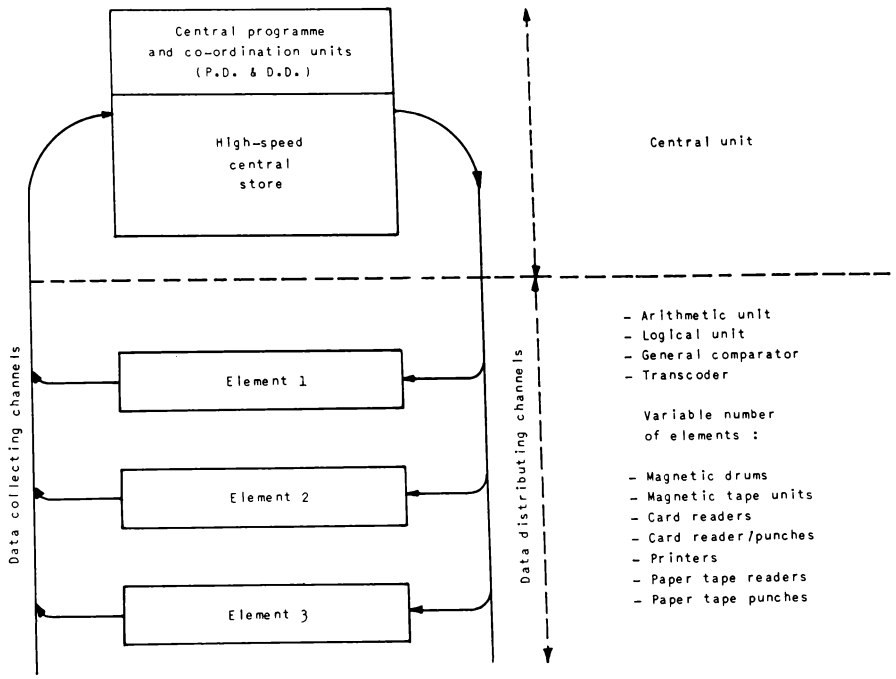
The channels are :

- the data distributing channel, which ensures the transfer of data from the central store to the elements.

- the data collecting channel, which ensures the transfer of data from the elements to the central store.

In addition to the central store, the central unit includes the central programme unit, as previously stated. However, since the elements can work independently, it is clear that in addition to the normal part this programme unit plays in the machines, it must also co-ordinate the operation of the elements. The programme unit includes the *programme distributor* (P.D.) and the *data distributor* (D.D.), the exact functions of which will be described further on.

The general structure of the Gamma 60 is as shown in the following diagram :



III - The elements : their connection with the central unit

In the Gamma 60, the term "element" is applied to any component or set of components which is able to operate independently once it has been given :

- instructions telling it what to do,
- instructions giving it the central store address of the operands.

1. As a rule, each element comprises two parts :

- *processing circuits*, in which data processing takes place (e.g. the addition circuits of the arithmetic unit),
- *control circuits*, which control the autonomous operation of the element.

The programme parts of the Gamma 60 are thus highly decentralized. First there are the central programme units which carry out directly certain instructions concerning the central unit itself, and transmit to the elements all instructions (directive instructions) pertaining to their autonomous operation; next, there are the control circuits of each element, which receive and interpret these instructions, and control the operation of the now independent element.

2. An element is said to be *active* when it undertakes a programme sequence. The central store then provides it with data to be processed and instructions for this purpose. However, it is the element which takes the initiative concerning any exchanges with the central unit, for instance :

- at the end of an autonomous operation phase, the element sends to the central unit a demand for the transfer of an instruction (D.T.I).
- when the element has received the necessary instructions from the central unit, it enters upon a further phase of autonomous operation during which data transfers, to and from the central store, are necessary : the element receives data from the central store and sends back the results, catena by catena. These exchanges are also made on the initiative of the element : each time a catena is to be transferred, the element sends a demand for the transfer of data (D.T.D.).

3. Since several elements can work simultaneously, and each on a different programme sequence, a "register" must be allotted to each element to memorize the central store address of the next instructions to be demanded, at the end of the current phase of autonomous work. This is called the *Instruction Address Register* (I.A.R.).

Each element therefore has an I.A.R. and all active elements have an address in their I.A.R. The I.A.R. of all the elements are grouped in the central unit in a matrix of magnetic cores (similar to those of the central store). Each "column" of the matrix forms an I.A.R. and consists of 15 cores, since an I.A.R. contains an address between 0 and 32,767 expressed in binary. This requires 15 bits.

The elements are set in action by special "cut" instructions, whose function it is to supply the I.A.R. of the element with the address of the instruction which follows the "cut".

4. Also attached to each element are a variable number (1 to 4) of *Data Address Registers* (D.A.R.) which contain either the C.S. addresses from which the element is to obtain the data for its work, or the addresses to which it is to send the results. Before the element starts operating autonomously, the registers are loaded with these addresses by means of special instructions called address instructions. The registers are similar to the I.A.R. in that they have 15 binary positions and are grouped in the central unit in a core matrix.

They are also known as *"current address registers"*, because each time data is

transferred following a D.T.D. from the element, the D.A.R. concerned is increased by one so that it always contains the address of the next catena to be transferred.

5. Each element has a *buffer register*. This component is essentiel for C.S. transfers to and from elements. C.S. transfer rate is 1 catena every 10 microseconds ; element transfer rate is much lower. For instance, the magnetic drum reads or writes one catena in 100 microseconds. Drum transfer rate is therefore ten times lower than high-speed store transfer rate.

It therefore becomes necessary to have a "buffer" or "transfer controller" component between the element and the high-speed store. This is the function of the buffer register. On the one hand, one catena is transferred to/from the high-speed store in 10 microseconds, on the other hand this catena is transferred to/from the actual element at the speed of data flow of this element.

For instance, in a C.S. - to - drum data transfer, the buffer register of the drum is fed in the course of one C.S. cycle, but the data with which it is supplied takes 100 microseconds (i.e. 10 C.S. cycles) to reach the drum. As a result, when transferring a certain number of catenae, to and from the drum, the data distributing channel will be involved during 1 cycle only out of 10. The 9 unused cycles remain available for other elements which may be active at the same time as the drum.

6. While an element is operating autonomously, it may have to produce qualitative results (such as from a comparison) in addition to, or in place of, quantitative results. Occurrences, such as overflow, may also arise ; such incidents are detected by the element itself, but, as is the case with qualitative results, they may bring about changes in the subsequent evolution of the programme. All qualitative data is memorized in a certain number of one-bit stores allotted to each element.

When the element sends out a D.T.I., these data are gathered together in a *"qualitative catena"* of the element, which, in turn, is sent to a specialized register of the central unit : the analysis register (or register A). The nature of this qualitative data is tested in this 24-position register by branch instructions whose function is to interrupt, conditionally, the sequence of programme steps according to the contents of the positions tested.

IV - Co-ordination of the operation of the elements

The description of the connection between an element and the central unit did not mention the problem of the operational co-ordination of simultaneously active elements. The three types of conflict arising from such simultaneity will now be examined, together with the means at the disposal of the Gamma 60 for solving them.

1. Channel clashes : data distributor

Since elements are autonomous and can operate simultaneously, it is not impossible for several elements to make simultaneous requests for data transfers. However, it is impossible for the central store to exchange several catenae with different elements in the course of a single cycle. Data transfers to and from the various elements requesting them are therefore made in successive cycles and in an order of priority which is dependent on the nature of these elements.

In order to solve this problem, each element sends out its D.T.D. along its own channel, and all the D.T.D. channels of the various elements lead to the *data distributor*. This component consists essentially of priority circuits whose task it is to

choose which D.T.D. will have priority, according to a pre-established order. For instance, if a magnetic drum and the arithmetic unit make simultaneous requests for data transfers, the drum will have priority over the arithmetic unit. It is clear that if the drum is kept waiting, it may lose a complete revolution, i.e. 20 ms., whereas if the arithmetic unit is kept waiting, it will lose no more time than the 10 microseconds it normally has to wait.

The data distributor thus ensures the simultaneous operation of the elements through the correct distribution of channel availability and of data exchanges with the high-speed store.

2. Clashes in the central programme unit : programme distributor

As is the case with the D.T.D., several elements may send simultaneous requests for instructions (D.T.I.) to the central programme unit; but the latter can only supply one element at a time.

The central programme unit is therefore equipped with priority circuits, into which lead one D.T.I. channel per element. When several requests are sent at the same time, one of them is selected according to a pre-established order of priority, as with the D.T.D. The central programme unit then works for the priority element until the latter has received all the data required and can start its phase of autonomous operation. The central programme unit is then free to send data instructions to another element.

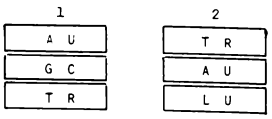
The priority circuits thereby enable the central programme unit to send out successively instructions to several elements and these elements are then able to operate *simultaneously and autonomously*. It is for this reason that the central programme unit is called the *programme distributor* (P.D.). The operational simultaneity of the elements is possible only because the time required for the P.D. to deal with the D.T.I. of one element is, on the average, very much less than the time required for the autonomous operation of this element. For instance, a magnetic tape unit is linked to the P.D. for about 100 microseconds, and reads a block in about 50 milliseconds; the channels are, moreover, occupied for only 5% of these 50 ms.

It should be noted moreover, that the programme distributor must, for the purpose of its work, call for transfers with the central store (where the programme is recorded), and may then clash with the autonomously operating elements. It is for this reason that the P.D. also operates via the data distributor when transfers with the central store are involved.

3. Clash between elements : recall chain

Usually, each of the various sequences with which the Gamma 60 deals simultaneously, makes successive use of the different elements : one, for instance, may use the arithmetic unit, the general comparator, and the transcoder in that order; another sequence may use the transcoder, the arithmetic unit and the logical unit. Each sequence section which uses only a single element starts off with a cut instruction calling for that element and is referred to as an "elementary sequence".

It will be noted however, that some elements belong to both sequences. Since, at a given moment, an element can work only for one single sequence, it must therefore work successively for the various sequences which have to use it.



However, the progression of the sequences is not synchronized. Consequently, it may quite well happen that sequence 2 sends out a request to the arithmetic unit while the latter is still working for sequence 1. This entails waiting until it has finished with sequence 1 before sequence 2 can be dealt with. However, the I.A.R. of the transcoder must be immobilized in order merely to keep track of the point reached in sequence 2, since the transcoder would then no longer be available for work on a possible third sequence. The sequences would be blocked successively and, in certain cases, there could even occur an unsolvable interlocking (e.g. with arithmetic unit - transcoder in

one sequence and transcoder - arithmetic unit in the other.). It is obvious that this clash must be overcome, otherwise it would not be true to speak of sequences being performed simultaneously.

The solution to the problem lies in the automatic operation of a system known as a *recall chain*. A detailed description of how this system works does not come within the scope of this brochure. It should however be mentioned that it consists in using two registers in addition to the I.A.R. of each element for the temporary storage of waiting sequences; one of these registers contains the address of the first sequence waiting, and the other, the address of the last one. As far as programming is concerned, the recall chain requires a blank catena after each cut instruction. The address in the second temporary storage register is thus always that of a blank catena. Each time the same element receives a request from a further sequence, the address of the latter is recorded in this blank catena and in the second additional register, as the address of the last sequence waiting.

Each new sequence thus brings with it a new temporary storage register for the next sequence, which means that the process can continue indefinitely and at low cost. Every time the element has finished with a sequence, it passes on to the first of the sequences waiting and continues in this way until the recall chain is empty.

4. Need for non-simultaneous operations : the virtual elements

It may thus be seen that the Gamma 60 achieves *self-ordination for the operations* entrusted to it, by means of the data and programme distributors and the recall chain. The only point at which simultaneity ceases is at the saturation point of the channels and P.D. Given the transfer rate of magnetic tape, it is not possible to read or write 20 tapes simultaneously ; there would, moreover, be no point in this happening, although more than 20 tape units may be connected to the Gamma 60 depending on the problem to be solved.

This permanent possibility of running several sequences simultaneously is troublesome in some cases. Trouble arises when, for economy's sake, two different sequences refer to the same data stored in the same work zone in the high-speed store. In these cases, sequences which have their instructions together in the central store, must not be allowed to run simultaneously. Similarly, when several simultaneous main sequences are likely to refer to a single independent subroutine, arrangements must be made so that this subroutine operates in connection with one sequence at a time, without interference from the others.

To do this, each sequence is coupled to a single virtual element. The sequence starts with a virtual element cut instruction, the purpose of which is to place an "engaged" sign in the catena whose address is given in the instruction. This catena is known as the virtual element image. A liberation instruction is given at the end of the sequence to be protected by a virtual element. As with the elementary sequences which cannot take place simultaneously when they use the same real element, the sequences which are associated with the same virtual element necessarily occur one after the other. A virtual element recall chain system, in all respects similar to the real element recall chain, may even be formed.

V - Checking in the Gamma 60

Internal checking in the Gamma 60 covers all work phases, from the input of data to the output of results. This gives rise to two distinct types of problem : data transfer checking and element operation checking. The latter point will be dealt with when describing each element individually, in so far as the principles on which checking is based differ from those of the transfer checking described in this paragraph.

In all transfer operations, each catena is accompanied by a check digit which makes sure that none of the data involved in the transfer has been distorted. The check digit has 3 bits. Each catena consists, in fact, of 27 bits - 24 bits containing the data and 3 for the check digit. Each catena is recorded with its check digit in all stores of the Gamma 60 :

- each central store block is formed of 27 core layers, 3 of which contain the check digits,
- each catena recorded on a magnetic drum track is followed by its 3 check bits,
- it was seen earlier that on magnetic tape, 6 of the tracks are used by the catena over 4 consecutive columns, the remaining 2 tracks being used for checking purposes. The check digit corresponding to a given catena is to be found on one of these tracks opposite the first three columns of that catena, (the space opposite the fourth column remains unused). The eigth track is used for a second check made by the tape itself, and which backs up the check digit. It contains a parity check bit for each column of the tape.

The check digit of a catena (three binary positions) is made up of two parts : the first, consisting of 1 bit is merely a parity check bit for the 9 most significant bits of the catena ; the second part, 2 bits, is the remainder of the division by 3 (carried out in binary notation) of the 15 least significant bits of the catena considered as being a 15-bit binary number. This division of the check digit into two parts is due to the fact already mentioned in chapter I, namely, that most elementary instructions consist of two parts of 9 and 15 positions respectively. When processed in the central unit, those two parts are dealt with separately and the division of the check digit into two parts greatly facilitates the checking.

Checking data transfers is very simple : the catena is transferred together with its check digit. A second check digit, similar to the first, is then made by a check digit generator and compared with the transferred check digit, to which it must be equal.

VI - Conclusion

This chapter dealing with the logical construction of the Gamma 60 may be found somewhat difficult and abstract. As a conclusion, we will propose an analogy, which, of course, cannot be taken too far but may possibly help to crystallize the contents of the chapter.

The Gamma 60 can be compared to a large machine workshop, such as can be found in the metal industry. Its elements play a role which is similar to that of the machine-tool - operative units. The processing circuits of the elements are similar to the machine-tools, and control circuits similar to the operatives.

The programme distributor is equivalent to the shop organization department which distributes the work to each operative in turn. The central store is the store from which the operatives obtain their tools (instructions drawn up by the P.D.) and where they go to collect the parts to be machined (data). The data distributor is the man in charge of the store, who distributes tools and parts to each operative in turn.

When, in a workshop such as this, a machine-tool - operative unit has received its working instructions from the work organization department, it becomes autonomous and, quite independently of the rest of the shop, the operative goes to the store to get the material he needs for his work, carries out his job on his machine-tool and then goes back to the work organization department to ask for further work.

The Gamma 60 element behaves in the same way : after instructions are received from the programme distributor, the element gets the operands required for its task from the central store, carries out this task and then asks the programme distributor for further instructions.

3

Elementary instructions

I - The different types of instructions : code B

In order to carry out a given piece of work the following four questions must be be answered :
- Which element is to do the work ?
- In what part of the central store are the operands and where is the element to store the results of its work ?
- Which are the operations to be performed?
- Under what circumstances is the job to be carried out ?

These questions are answered by four types of instructions :
- Cut instructions (C instructions), which activate the element needed.
- Address instructions (A instructions), which give the address of the operands.
- Directive instructions (D instructions), which specify the job to be carried out.
- Jump instructions (B instructions), which modify the usual sequence of instructions, and thus provide an answer to the last question above.

As mentioned in chapter I, these instructions expressed in internal machine code (or code A) are stored in one catena. When the programme is to be run, they are placed in the high-speed store and are carried out in ascending order of address, except in the event of jumps.

All sequences must start with a cut instruction. All instructions written between two cuts for a real element refer to the element indicated by the first cut. An *elementary sequence* is a group of instructions which begin with a cut and end at the next cut.

An element can start autonomous operation once it has received a *directive instruction* and when its local programme has interpreted it. Once the directive is executed, the element calls the programme distributor (P.D.) for more instructions. As soon as it is available, the P.D. deals with the next sequence of instructions in the programme and executes them until it finds a directive instruction to send to the element calling. The P.D. will not answer any other request until such a directive instruction is found. Directive instructions can be said to mark the limits of *complete instructions* just as cut instructions mark the limits of elementary sequences.

The four types of elementary instructions are described in this chapter. No reference is made to the binary internal code (A) described in chapter I. A mnemo-technical code, known as code B, will be used. There is a one-to-one correspondance between code A and code B instructions.

II - Structure of an elementary instruction in code B

An elementary instruction in code B has a fixed structure of 13 alphanumerical characters. Most instructions do not use all of these 13 characters ; the unused ones are left blank.

The 13 characters are divided into four groups. Each group corresponds to a zone on the code B programming sheet. Note : code B is not the only programming possibility in the Gamma 60. A more comprehensive code, referred to as code C, will be found in the programming brochure.

1. Type of operation : (T.O.). A 4-character zone indicates the type of operation.

All four characters are used for cut instructions. A 3-character code, set on the right-hand side of the zone, indicates a directive, which starts the autonomous operation of an element. Jump instructions and certain instructions known as *central directives* (thus named because they are not sent to elements but are carried out directly by the central unit) are represented by 2 characters, again set on the right. Address instructions are represented by a single character in the righthand column.

T.O.				C.C.		X					A.F.	
C	U D	T I C	I R D A					1	4 8 2	0 2 8	R R	S S

Structure of code B

This system corresponds exactly to the order of importance of the instructions. The code B sheet thus shows clearly, first, the cuts, then the directives for elements, which mark the complete instructions, and lastly the jump instructions, central directives and address instructions which do not release the programme distributor.

2. The next group consists of 2 characters, called *complementary codes* (C.C.). which are used in various ways according to instructions.

3. The third group is formed by 5 characters and is named the *free variable* or *address zone* (X). In most instructions, the 15 righthand side bits represent an address, a length, or an element number expressed in pure binary digits. In code B, this quantity (32,767 maximum) is expressed in decimal notation in the "free variable" zone. All 5 characters are not usually necessary and the number is then set on the right.

Some directives (especially those for the arithmetic calculator) do not indicate operand addresses directly ; they are instructions with three indirect addresses, and these are given in the free variable zone, as will be seen later.

4. The last 2 characters are concerned with the *address function* (A.F.), the task of which is to modify, if need be, the address section, if it exists (consequently, directives for the arithmetic calculator have no address function).

This address function is an essential feature of the Gamma 60 instructions, and played a leading part in the preparation of programming methods. When a programme segment, a sub-routine, is written in code B, it is not known, in advance, in which part of the high-speed store it will be located when it is about to be carried out. Sub-routine instructions are therefore programmed in *relative* form and numbered from 0 to n.

As a result, in a jump instruction written on line 5, referring to line 15 of the sub-routine, the figure 15 is written in the free variable zone of the jump instructions and the letter "R" placed in the first column of the address function :

Br 15 R

(Br is used for Branching)

In internal code (code A), this is expressed by a 1 in position 17 of the catena, while the difference, 15 - 5 = 10, between the address referred to and the address of the actual instruction is shown in the address section. This difference remains constant, regardless of location in the high-speed store. Therefore, when jump instructions come to be examined by the programme distributor, the latter, knowing the address of the instructions concerned, refers to an address which is 10 catenae further on.

Such is the *differential address* function. It will be noted that negative differential addresses (jump instructions referring further back, for instance) are expressed in code A by the complement of the difference of the addresses to 37,768.

However, many instructions make it possible to store, in any given store, a length or an address to be used later on. Instructions wishing to use these stored addresses must be written as follows from this example of a branch instruction :

"Jump to the address contained in location 2.348". 2,348 is written in the free variable zone and letter "S" placed in the second column of the address function. In code A, this is expressed as a 1 in position 16 of the catena. When the programme distributor finds this 1, it replaces the 2,348 read in the address section by the contents, of store 2,348.

Such then is the function of *address substitution.* The two address functions- differential and substitutional - often occur together. In the above example, there is no reason for the absolute address of the storage register to be known when writing the programme. This register is often placed at the end of the sub-routine, and only its relative address in the sub-routine is known. The instruction is then written :

Jump to 29 R S

assuming that the 29th line of the sub-routine contains the address to which the jump is to go.

When the programme distributor finds this instruction, it carries out the differential function first, and then the substitutional function.

III - Cut instructions [1]

These are genuine flowchart instructions. In addition to the fact that cuts on real elements mark elementary sequences, all cuts may be used to mark the point where simultaneous independent sequences must meet before starting one single sequence. A special cut can also be used to launch two simultaneous sequences from a single one ; another cut ends the sequences. Finally the cuts on virtual elements ensure that the various sections of the flowcharts are performed in their appropriate order.

To explain this, the different types of cut instructions are analysed as follows :

1. Cuts on real elements.

They cause the element requested by the cut to start operating and they release the requesting element. They may cause the recall chain mechanism to start, in so far as it is required. The 4 letters which correspond to the type of operation show which sort of element is being called, e.g. ARIT for the arithmetic unit ; element numbers are given in the free variable zone, only when it is possible to have several elements of the same type e.g. IMPR.

2. Cuts on virtual elements

PROT (protection) engages the virtual element and LIBE liberates it. Subject to the address function, the address of the virtual element image is to be found in the free variable zone.

(1) The code B table of elementary instructions, appearing immediately after the conclusion on page 53, will help to illustrate the following paragraphs.

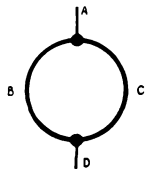
3. STOP and SIMU

STOP ends a sequence (nothing in free variable zone) and SIMU launches simultaneously :

- the sequence which normally follows this instruction,
- a 2nd. branch of programme, the address of which is given in the free variable zone of SIMU.

This latter instruction enables flowcharts to be programmed in their natural state, which is usually a "network".

When a sequence A produces two independent sequences B and C, it is necessary, when working with sequential-programme machines, to make the flowchart sequential too, by carrying out B and C one after the other in any order before carrying out D.



This procedure does not apply to the Gamma 30. A SIMU is placed at the end of sequence A, and B and C are carried out simultaneously. The end of B and C both refer to the 1st instruction of D which is a special cut instruction, a 1 being written in the complementary code and showing the number of additional conditions expected before D is to be started. In this case, the cut will come into effect only after it has been called twice by the programme distributor - once at the end of B and once at the end of C. 1, 2 or 3 is therefore written in the complementary code, depending on whether the cut is to wait for 2, 3 or 4 sequences. If a cut is placed in the course of a sequential programme, there is, of course, no complementary code to be written.

IV - Branchings

Their purpose is to break the sequence of programme steps in order to jump to an address which is given in the free variable zone of the actual branching.

There are two types of branchings - the systematic and the conditional.

Systematic branchings : the T.O. of the "always" systematic branching is 31.

The T.O. of the "never" branching is 00. This instruction is ineffective. When the programme distributor comes across it, it merely passes on to the next instruction. However, it is necessary to include occasional instructions of this sort in the programme (for their address sections and functions) since the programme may transform them into "always" jumps, according to the circumstances.

Conditional branchings : their T.O. is coded from 01 to 24. These may or may not be effective, depending on whether there is a 1 or a 0 in the analysis register A of the central unit at the position given by their T.O. It is usually the qualitative catena of the active element which is thus tested. However, a central directive allows any catena in the central store to be transferred to the A register. In this way, any bit of a central store catena may be tested by means of a conditional branching.

V - Address instructions

They are the same for all the elements and concern the element indicated by the preceding cut. There are four address instructions :

- Load a CAR (T.O. : L) with the address written in the address section of the instruction.
- Place (T.O. : P) the contents of a CAR in the address given by the instruction itself.
- Add to (T.O. : A) and subtract from (T.O. : S) a CAR, the quantity given in the address section of the instruction.

The CAR number (0, 1, 2 or 3) of the element to which the instruction refers is given in the first complementary code.

The address section of these instructions can, of course, be modified by the A.F.

VI - Directives

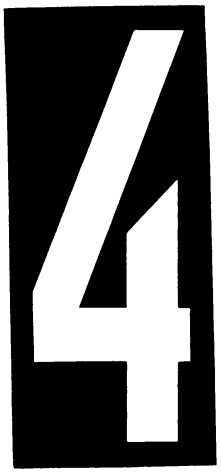
As mentioned above, there is a difference between the element directives which release the programme distributor (P.D.) and which are coded by a 3-character T.O., and the central directives which are carried out in the central unit without releasing the P.D. and are coded by a 2-character T.O.

The structure of the former varies with the elements and will be dealt with at the same time as each of these elements.

The function of the latter is to carry out a certain number of logical operations concerning calculations and instruction modifications which often turn up in programmes. These operations could all be carried out by the logical calculator, but it was found to be useful, in some cases, to have central, high-performance operations.

A complete series of instructions executed in a register B in the central unit is to be found amongst them. This 24-position register is a genuine operator-register. In the 15 righthand positions it is possible to carry out the same operations as in CAR (loading, addition , subtraction) without making use of the 9 remaining positions. This is obviously of great advantage for instruction calculations.

There are also highly synthetic instructions such as iteration control (I.C.), which checks the number of repeats to be run in a programme loop. The counter progression (C.P.) makes it possible to use any location in the central store as an ordinal binary counter.



Data processing units



Systematic maintenance of the "cabinets" of the Central Unit. Note the sliding doors, allowing free access to the different circuits.

I - The arithmetic unit

The arithmetic unit operates only with semi-logarithmic numbers, as has already been mentioned. Results are also expressed in semi-logarithmic form, with either a floating decimal point or a fixed decimal point (position of the decimal point set by the programmer) depending on the code to be found in the first complementary code of the directives. Prior to any operation, the arithmetic unit automatically shifts operands according to the value of their exponents.

1. The arithmetic calculator has 3 working circuits (for sign, mantissa and exponent) ; they operate automatically and the programmer never has to intervene. It also comprises 3 registers of 2 catenae each (because each number in semi-logarithmic form requires two catenae) which are accessible for programming purposes :

- The P (principal) register, which receives the first operand at the beginning of each operation and the result at the end of each operation.
- The S (secondary) register, which receives the least significant digits (if the result exceeds 10 digits - e.g. the multiplication of a 10-digit number by a 10-digit number). The exponent of the number in S is 10 lower than that of the number in P. Registers P and S form the accumulator of the arithmetic unit.
- The O register, which receives the second operand at the beginning of each operation.

In fixed decimal-point operations, the position given for the decimal point of the result is understood as being "in the P register" only. It is written in the first two columns of the free variable. Overflow may then occur and is reported in the qualitative catena.

2. Checking of the arithmetic unit. Operand transfers from the central store to the arithmetic unit, or result transfers from the arithmetic unit to the central store are, of course, checked catena by catena, by means of the 3-bit catena check digit. Once the operands have been transferred to the arithmetic unit, checking is taken over by a local check system consisting of a decimal check digit obtained through division by 7. A check digit is formed for each operand (2-catena decimal number), and consists of the decimal remainder of its division by 7. This operation is then carried out on the actual numbers and on their check digits; the check digit of the results is then compared with the result obtained from the similar operation on the operand check digits : they must be equal.

This system of "cast out of the 7" is exactly the same as the "cast out of the 9" operation taught to school children. It is, however, far more reliable. "Cast out of the 9" does not disclose digit inversion in the results : 2,548 and 8,452 for instance, both have the same decimal check digit when divided by 9. The division of these two numbers by 7, however, leaves remainders equal to 0 and 3.

3. Arithmetic unit directives. These instructions have no free variable and therefore no address function. They have three indirect addresses : the three righthand columns of the free variable zone denote the locations of the 1st operand, the 2nd operand, and the place where the result is to be stored.

The arithmetic unit has four non-specialized current-address registers which progress automatically when operands are transferred to/from the arithmetic unit. They can be reset to their initial value at the end of each operation; for this purpose, they are not designated by a digit (0, 1, 2 or 3) but by the initial letter of the digit (Z, F, S or T). This makes for great versatility in calculation programming.

The arithmetic unit naturally carries out the four operations; it also carries out several minor operations which may be found in the table of directives, such as the comparison of algebraic numbers, the results of which are given in the qualitative catena :

positions :	1,	2,	3,	4,	5 and	6
result :	>	<	=	>	<	≠

The arithmetic unit always uses 2-catena operands, i.e. a maximum of 10 significant digits. However, it is possible to obtain a double-length result, i.e. the entire content of the accumulator (P register + S register) through the use of the directive's second complementary code. The result has then 20 significant digits.

It is possible to carry out calculations on operands of more than 10 digits with the help of "double precision" sub-routines.

4. Operating times :

Addition, subtraction	: 200 microseconds
Multiplication	: 350 microseconds
Division	: 550 microseconds
Comparison	: 150 microseconds

These times include :

- the transfers of operands,
- the setting of operands and results according to decimal point position, in either floating or fixed decimal point notation,
- actual operating time.

II - The general comparator

1. The essential function of the general comparator is, obviously, the comparison of quantities, either numerical or alphanumerical, in order to classify, collate and sort them.

Variable length operands (their length is given in the free zone of the directive as a number of catenae) are compared catena by catena. Each catena is considered as a pure, 24-bit binary number. Comparison is also valid for the coded contents of the catena (6 digits or 4 letters), since the internal code was chosen so that a 6-digit binary number, any alphanumerical character, is greater for Z than for A. Comparisons start from the left "most significant figures first"; they stop as soon as a difference is detected. They continue to the end of the designated length only if equality is found all the way.

It should be kept in mind that comparisons between semi-logarithmic numbers are performed only by the arithmetic unit, owing to the preliminary shifting required.

2. There are two types of comparisons which correspond to two different directives:

- one-way comparison, which is a comparison between two quantities;
- two-way comparison, which locates a variable quantity in an interval defined by two fixed quantities. This function could, of course, be performed by a series of one-way comparisons; the two-way comparison is only a more comprehensive operation. Results are given in the qualitative catena in the form of tests meaning :

- smaller than limits,	$< L_1 L_2$
- greater than limits,	$> L_1 L_2$
- between limits, including limits,	$L_1 \geqslant x \geqslant L_2$
- equal to one limit,	$= L_1$
- equal to the other limit,	$= L_2$
- between limits, excluding limits.	$L_1 > x > L_2$

3. Two secondary functions have also been assigned to the general comparator.

a) it has already been stated that there are no direct central-store to central-store transfers in the Gamma 60, and that such transfers must be made via an element. Provision has been made for the passage of rather long *transfers* through the general comparator by means of directives giving transfer length.

b) it may prove necessary to *delete a given central store zone*. The general comparator directive then indicates the length of the zone to be cleared.

4. Addresses of the terms in comparisons, of the origins of transfer zones and of zones to be cleared are held in the current-address registers of the general comparator. This element has 3 specialized current-address registers, all of which are used only in two-way comparisons. Current-address registers 1 and 2 contain limit addresses, and current-address register 0 contains the address of the variable quantity. Only current-address register 0 is used for deletion and is implicit in the corresponding directive Ine one-way comparisons or transfer operations, current-address register 0 always contains the address of the first term, and current-address registers 1 and 2, that of the second. The register number 1 or 2 must be specified in the first complementary code of the directives.

5. One-way comparisons and central-store to central-store transfers take 30 microseconds per catena compared or transferred ; two-way comparisons take 50 microseconds per catena, and deletion takes 10 microseconds.

III - The transcoder

1. Three functions have been assigned to the transcoder :

a) The *transcoding* of external codes (punched cards or paper tape) into the Gamma 60 numerical or alphanumerical code, and, inversely, the transcoding of internal code into external code for data output on punched cards, punched paper tape or through the printer.

b) The *distribution*, or *editing* of data. The Gamma 60 is not equipped with plugboards on input-output units. However, printed data still needs to be edited according to a certain layout. This task is undertaken by the transcoder. While transcoding is going on, data are stored in the central store in the order required and the necessary spacing provided. The image corresponding to the final edited version of the document for output (punched card, punched paper tape or printed data) can thus be obtained from the central store.

c) the condensing or expanding of data. Data, expressed in internal code, can assume two forms :

- one is for *processing* purposes. Here the words are represented in maximum theoretical length and are rounded off to a whole number of catenae. Data thus set out is ready for processing and there is no mixing of numerical and alphanumerical codes in any one catena.
- the other is for *storage* purposes. Here words are in their true length and follow each other in the catenae. This process is used for the minimum volume storage of data, e.g. on magnetic tape (data prepared for processing is also stored on magnetic tape and it proves very often to be the most convenient form).

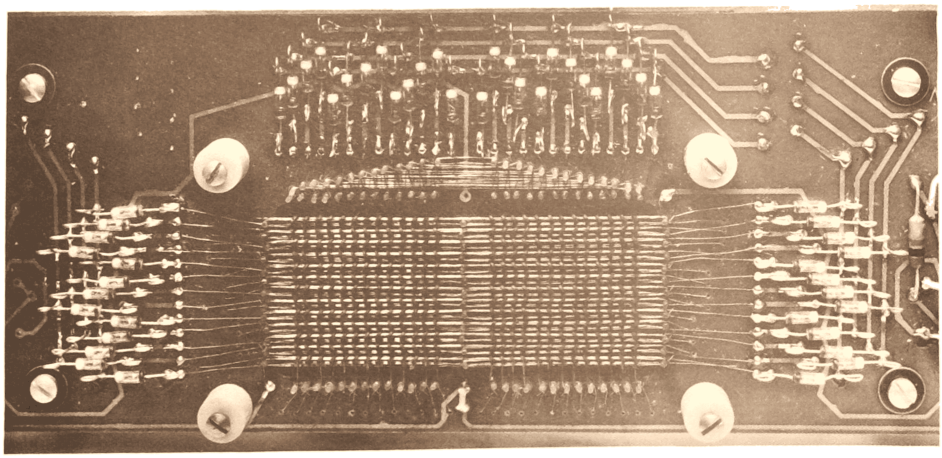
A further auxiliary activity of the transcoder is line-to-column transposition. Cards are read or punched line by line, but must be transcoded column by column, since each column holds a character.

The transcoder also puts into semi-logarithmic form the numbers which are to be processed by the arithmetic unit.

2. The transcoder processes each character separately. The way in which it operates varies owing to the layout complexity of incoming cards or of outgoing printed data. The extent of these variations is such that changes may even occur for each character, one being expressed numerically, another alphanumerically, yet another in pure binary digits, etc. The programme distributor would indeed be flooded if the transcoder had to ask for directives on each occasion. For this reason the transcoder programme is divided into two parts.

a) the *actual programme* which gives the "phase" directives, i.e. those which inform the transcoder that it is to translate punched card data into internal code using the processing form or storage form punched paper tape data or internal code into printer code, etc. There are nine "phase" directives available. These directives lay down general operating conditions for the transcoder and are followed by a "Refer To Cliché" (R.C.L.) directive which starts the job according to the translation cliché.

b) the *translation "cliché"*, which directs the work of the transcoder, character by character. This translation cliché consists of "cliché codes" at the rate of 4 per catena. There are "positioning" clichés which inform the transcoder whether it is going to translate a numerical word, an alphanumerical word, an algebraic number, etc. and "execution" clichés, which cause the work to be carried out on a given number of fixed characters. Lastly there is a "return to programme" cliché which is self-explanatory. The distinction between the translation cliché and the programme lies in the fact that, following a D.T.D., the former is supplied catena by catena into the transcoder, as though it were a quantity. It is left entirely to the local programme of the transcoder



Transposition matrix. This consists of magnetic cores, similar to the matrix for the interpretation of codes and the matrices of data address registers and instruction address registers. Its particular role is to change over from lines to columns during the reading of punched cards.

to interpret it. This avoids flooding the programme distributor but calls for a transcoder CAR (CAR n° 2) to keep track of the cliché. Incidentally, all CAR's have specialized roles : CAR 0 gives the address of the data to be transcoded, CAR 1 gives the storage address of the data transcoded.

3. The *principle of transcoding* is as follows : the transcoder has a plane, magnetic-core matrix divided into 256 addresses. Each address contains a character of the *code* which is *required*. The character in the *code* to be *translated* is expressed by the transcoder as a binary number which is the address in this matrix of the character in the code required. This matrix is known as the "matrix for the interpretation of codes" (M.I.C.). The principle of transcoding therefore consists of selecting, from the M.I.C., the address which corresponds to the character to be translated. However, special directives (M.I.C. loading) make it possible to transfer to the M.I.C. any table which has first been compiled or supplied by the central store. It follows that the Gamma 60 can translate *any external code :* Bull, I.B.M. or I.C.T. punched cards, 5 or 8-track punched paper tape into Bull, I.B.M. or any other code.

A translation operation does not usually make use of all the M.I.C. and several transcoding tables can therefore be stored simultaneously in the M.I.C. It is thus possible, without having to reload the M.I.C. for each operation, to carry out several simultaneous conversions, e.g. :

- Bull cards to magnetic tape,
- magnetic tape to I.B.M. cards,
- punched paper tape to magnetic tape.

4. The internal *check* of the transcoder by the parity check digit which partners each character.

5. The exact calculation of translation times is highly complex and depends on the "phase" directives and the complexity of the editing required. However, *15 to 20 microseconds* per character translated may be considered as a fair approximation.

IV - The logical unit

1. The logical unit has two functions :

- to carry out the 4 arithmetic operations on pure binary numbers of a maximum of 24 binary digits (1 catena).
- to carry out logical operations for the sectional division and sorting out of data, "data filtering". With the logical unit it is thus possible to deal with data in the elementary bit stage, i.e. the most analytical stage.

It thus becomes evident that all central directives concerning instruction modification or complex operations such as iteration control could be carried out by the logical unit but under far less favourable conditions.

2. The calculator consists basically of two registers which can be used when programming : the K register and the L register, which are in some ways similar to the O and P registers of the arithmetic unit. These are shift registers with serial access and have 24 binary positions each. Results go to register L at the end of operations.

The logical unit has 4 specialized CAR's, which are used in exactly the same way as those of the arithmetic unit.

3. Directives are similar in structure to those of the arithmetic unit. They all have "filtering" or "shift" indications (columns V), but only one address (namely for storage of results) which is given directly by a CAR number.

Prior to all operations, operands must be supplied to K and L registers by two directives which carry out simultaneous setting, by means of the circular shift to the right of a number of positions equal to that given in V.

In addition to the directives which correspond to the 4 arithmetic operations, there are :

- instructions which enable catenae of any shape to be constructed from zero,

- instructions which make it possible to "assemble" a catena from several "pieces" of any length coming from different catenae,

- logical intersection or reunion instructions concerning whole catenae or fractions of a catena,

- in addition to the standard results $L > K$, $L < K$, $L = K$, etc., the following indications are given in the qualitative catena after the binary and logical comparisons concerning the logical structures of L and K :
  - inclusion of L in K or K in L,
  - exclusion of K and L (intersection would give 0 everywhere),
  - the reunion of K and L is all 1's; when in conjunction with the previous indication, this is characteristic of the complementary nature of the two structures.

5

Input-output units

These are the : - Card reader-punches
- Card readers
- Paper tape readers
- Paper tape punches
- Printers

Subject to the following provisos, up to 40 input-output elements can be connected to the Gamma 60 and distributed over the various categories as desired :

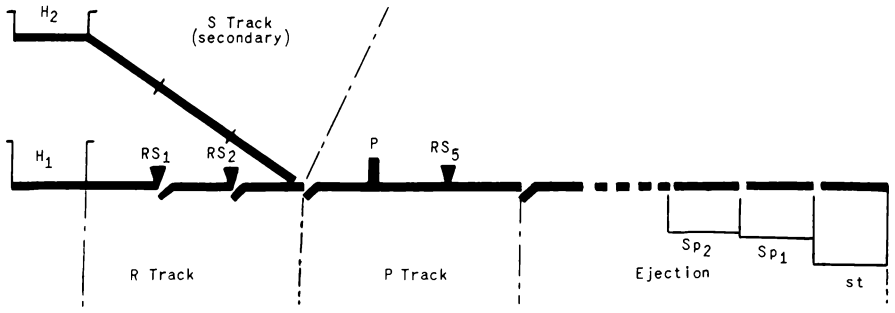
- Each reader-punch counts as two elements.
- There can be no more than 12 card tracks in all, and, of these 12, no more than 6 reader-punches.

I - The card reader-punch

1. The card reader-punch usually has one track for either reading punched cards, or punching cards at the rate of 300 cards per minute. It is impossible, however, to maintain this rate if reading and punching are to be done in different fields on the same card. A method of operation such as this would, moreover, be typical of a "card by card" job which should not come into the "integrated" work which the Gamma 60 can and should undertake.

A secondary track can be added to the main track, for feeding blank cards to be punched. When a main file is being read on the main track, it then becomes possible to collate it with another file which is punched on to the blank cards fed by the secondary track.. When thus used, the reader-punch output of punched or read cards into the card stacker still amounts to about 300 cards per minute. It should be noted, however, that punching and reading are here carried out on different cards.

2. The reader-punch track is made up as follows :



Feed hopper H1 can hold 3,000 punched cards to be read or 3,000 blank cards to be punched.

The reading track is equipped with two reading stations RS1 and RS2, which read the cards twice; reading is checked by comparing the two results.

The punching track has a line-by-line punch and a reading station, RS5, for checking punches.

The feed hopper of the blank card feeding track has a capacity of 800 cards. The cards are usually collected in a 2,800 card capacity stacker (st) at the output end, or in one of two 700-card special pockets (Sp1 and Sp2). One of these pockets receives incorrectly-read cards, the other selects certain cards according to the data they contain.

3. Reading and punching checks are carried out automatically by the reader- punch. Selection at the special pocket of incorrectly-read cards is programmed. It is for this reason that reader-punch directives also contain pocket-selection directives.

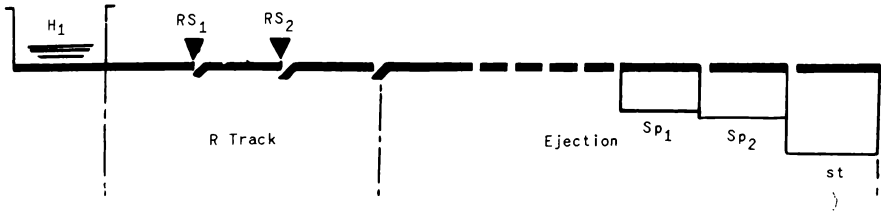
4. When operating normally, the entire track is in motion. Circumstances do, however, occur (e.g. when blank cards are inserted from the S track) in which only partial movement of the track is required. It may even become necessary to have cards move one cycle without any reading or punching operations.

The normal reading and punching directives, therefore, include a directive for moving the track without reading. In addition, complimentary codes in the directives are used when a split track is required and promote, if necessary, partial track movement between RS1 and RS2, RS2 and the point at which the secondary track joins the others.

5. Normal incidents (e.g. end of file) or abnormal incidents (e.g. track jamming), cause the machine to stop and are reported in the qualitative catena. This makes it possible to send, via the programme, a demand for normal manipulation (end of file - D.M.F.) or a demand for manipulating incidents (D.M.I.), which turn on signal lights both on the computer and on its general control desk, to inform the operator that his intervention is necessary. Such directives exist for all peripheral elements where the operator may have to intervene.

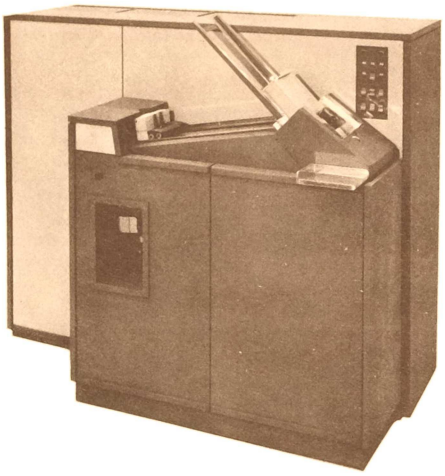
II - The card reader

The card reader also operates at the rate of 300 cards per minute and programming and card-reading capabilities are identical to those of the reader-punch, but it is not equipped with a punching track. Its structure is as follows :

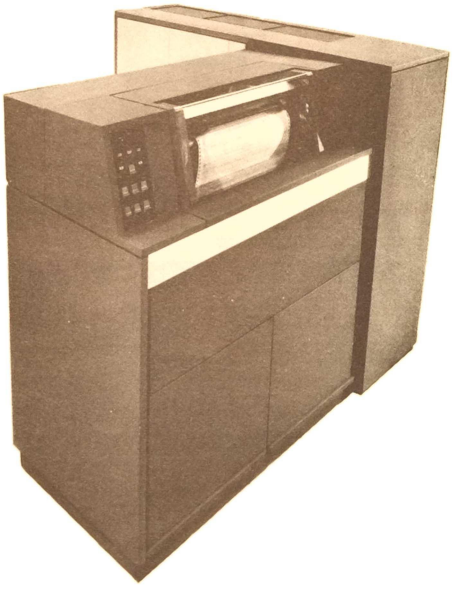


III - The paper tape reader

The paper tape reader operates at the rate of *300 characters per second.* Reading is done by photo-electric cells, which can read either the 5 or the 8-track tape, once the correct tape width has been set in the reader by a track adjuster.



Card Reader-Punch. To the right can be seen the card feed hopper inclined at an angle of approximately 45°; beneath are the reception hopper and the two vertical special pockets.



The Printer. The cabinet coupled to the print unit contains the electronic circuits attached to the Printer.

Tapes can be read both forwards and backwards.

The characters read are transferred to the central store at the rate of 3 characters per catena for both tape widths. The transcoder then translates them into internal code, since the tape reader, like the card reader, does not perform any transcoding, but merely transmits an accurate reproduction of tape contents to the central store.

However, with the 8-track tape, which has an imparity track, the tape reader itself verifies the characters read to make sure that they correspond to an odd number of punches. If faulty reading is detected, the reader stops before the next character is read and the faultily-read character is not transferred to the central store. The reader also stops if there is no tape (breaks) or at the end of the tape (no punching).

The length, in catenae, of the central store zone which is to receive the result of the reading is given in the reading directives.

IV - The paper tape punch

The paper tape punch operates at the rate of *25 characters per second* and punches the code prepared by the transcoder. Characters are transmitted to the punch in groups of 3, i.e. catena by catena, as in all central-store to element transfers. The actual punching is not checked.

Punching directives give the length of the central store zone reserved for the data.

V - The printer

This 120-position printer operates at the rate of 300 lines per minute. 59 different characters can be printed in each position (cf. Gamma 60 code chart).

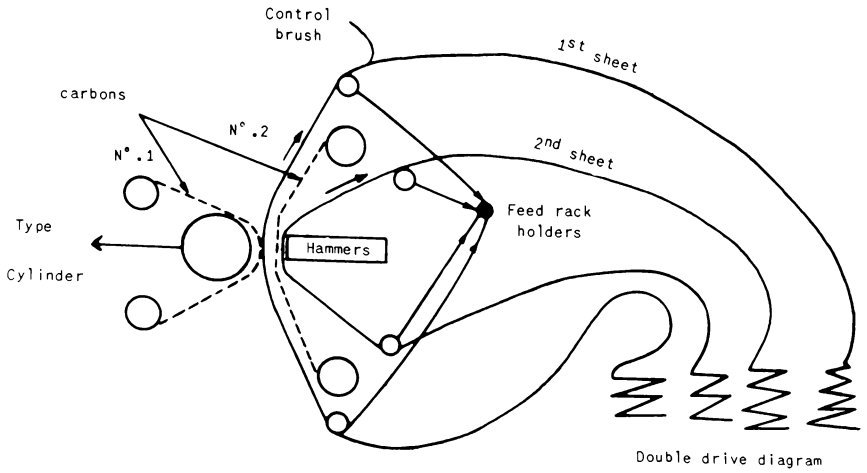
1. The principle on which it operates is as follows : the 59 possible characters are repeated 120 times on a printing platen and are arranged in such a way that the same character is to be found 120 times along a line parallel to the platen axis, and the 59 different characters are to be found in the same line around the circumference of the platen. This platen revolves about a horizontal axis at the constant rate of 300 r.p.m.

Sprocket wheels move the paper forward in a plane parallel to the platen axis. A row of 120 small printing hammers is behind the paper, and, as the character to be printed passes in front of it, the corresponding hammer presses the paper against the cylinder. The mechanical movement of the printing hammer creates an electric impulse which, together with the position of the platen makes it possible to ascertain that the correct character has been printed ("echo" printing check).

2. Although the printing drum has 120 spacings, paper size is 180 spacings wide and extends 20 spacings beyond the lefthand end of the platen and 40 beyond the right.

Type spacing is 1/8 th inch wide and *line spacing* 1/6 th inch.

Both margins of the paper are punched every 1/2 inch with small, round, "Carroll" perforations and the paper is driven by sprocket wheels. The sprocket-wheel racks can be moved sideways to allow for all widths of paper.

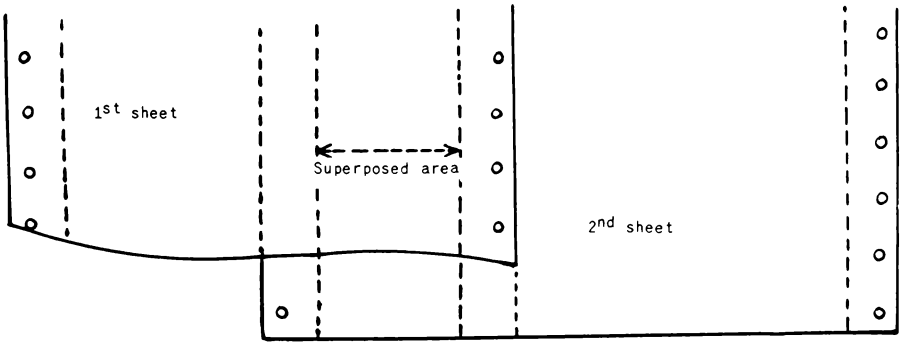


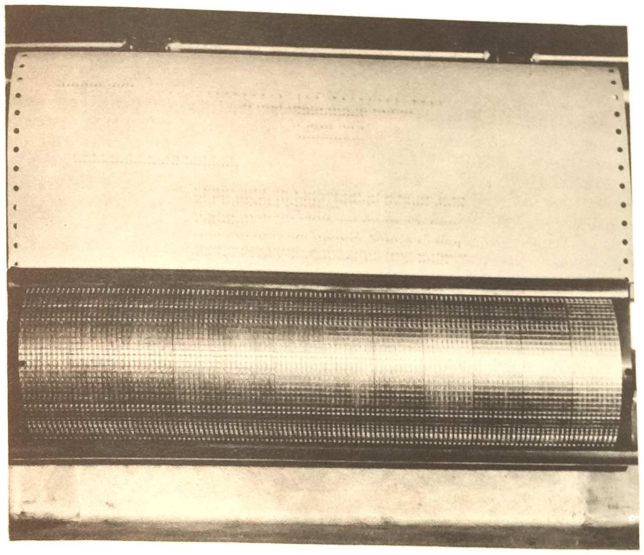
Double drive diagram

3. There are two paper-drive mechanisms which operate quite independently of each other. It is thus possible to duplicate printing either totally or partially, or in columns side-by-side. This arrangement is particularly advantageous for the simultaneous printing of receipts and lists. In addition, side-by-side printing doubles printer output by the simultaneous printing of papers of which the sum widths do not exceed 120 characters.

The double feed calls, of course, for *two carbons;* one is placed between the type cylinder and the first sheet, the other between the first and the second sheets. The carbon paper is the same width as the type cylinder; it is rolled on two drums and unrolls at the same time as the paper but at a lower rate. This rate can be adjusted from 0.5 to 3mm per printed line. The first carbon can be backed with material but the second one must have a paper backing.

Partial overlapping of printed matter





Print unit type cylinder

Continuous stationery can be placed on each drive but their total force must not exceed 350 gr. per sq. m. and their weight per drive must not exceed 150 gr. per m.

4. Each printing directive causes the printing of a line which has previously been edited in the central store by the transcoder in the phase P.P.R. - processing form to printer.

The free variable of the printing directive gives the number of line spaces to be skipped before the line is printed. *Paper skips are thus programmed.* However, these directives control only skips on the first paper. Skips on the second sheet (which are not necessarily the same, since the two paper drives are independent), are controlled by another special directive (second-sheet skip) which precedes printing directives.

In order to prevent the paper from slipping, *skips are checked* by rectangular perforations which are with the Carroll perforations in the drive margins. There is one check perforation per sheet.

When a "sheet reference" line is to be printed, an electric contact is made via the rectangular perforation by means of a metal brush and is reported in the qualitative catena; it can also be tested in the programme. The check perforation also enables the initial positioning of the paper to be automatic.

VI - The Gamma 60 control desk

This is not an input-output element, and it is not included in the list of elements given at the beginning of this chapter. However, since it consists basically of a typewriter, it can be looked upon as an input-output medium. It provides the means for a dialogue between man and machine.

The operator uses his typewriter keyboard to send orders to the Gamma 60 to start a programme, to modify given programme instructions, or to direct operations in the event of incidents which are not automatically corrected by the programme. Everything that the operator types is, of course, printed at the same time.

In the opposite direction, the typewriter prints information supplied by the Gamma 60 about any event that could call for a decision from the operator, e.g. end of programme, incidents, requests from the computer for orders from the operator, etc.

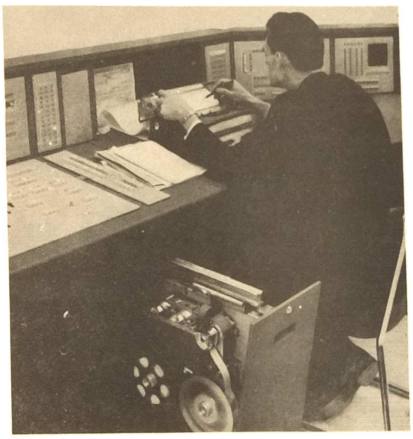
The typewriter at the desk thus makes a dialogue possible between the operator and the Gamma 60 while, at the same time, printing an account of this dialogue, which thus forms a true log-book. A Gamma 60 programme which is continually present in the central store conducts these exchanges ; this is the "typewriter management" programme.

This programme can, of course, launch an operation while another operation is already in progress. All operations are launched in the same way, whether they are the first to be started in the machine or whether they are started when others are already in progress. The only condition with which programme launching must comply is the availability of peripheral elements and of the central store zone required.

A paper tape reader and a punch are connected to the desk typewriter and are governed by its output rate. Control-desk - Gamma 60 exchanges can thus be recorded on punched paper tape and orders sent out from the desk to the Gamma 60 can be recorded on paper tape prior to their transmission.



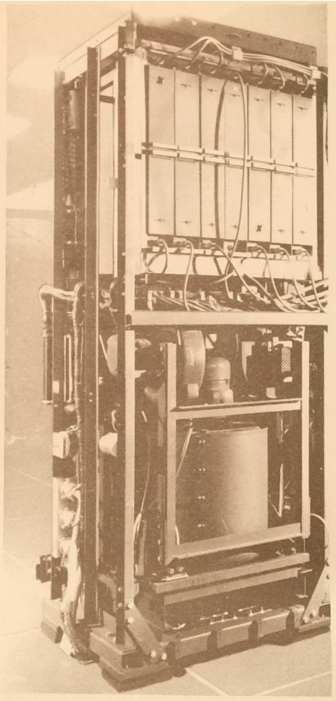
The Gamma 60 console consists mainly of a typewriter which is used either by the operator or by the Gamma 60 during the course of the work. Control panels at the right of the machine, make it possible to check the operation of the different elements. On the left may be seen a series of switches and buttons which are used solely by the team responsible for the daily maintenance.



Console (close up). In the open drawer in the foreground may be seen the paper tape punch, connected to the typewriter, allowing a complete record of Gamma 60 operations.

6

Data storage units



Magnetic drum with its read and write circuits.



Magnetic tape units.

These Data Storage Elements are the magnetic drums and the magnetic tape units. They have already been dealt with, to some extent, in the paragraphs on the Gamma 60 stores in chapter 1. We shall now consider them as "elements to be programmed".

I - The magnetic drum

Its features and performance are given in chapter 1. It is equipped with a C.A.R. which contains either the central store address of the data to be transferred to the drum, or the central store zone which is to receive data transferred from the drum.

The drum address at the other end of the transfer is in one of the local programme registers in the drum. It is not determined by address instructions (which concern only C.S. addresses) but by a special directive (D.A.O.) : Designation of the Address of the Origin.

The actual transfers are controlled by two directives, the T.O. of which give transfer direction and the free variable, transfer length. A maximum of 4 magnetic drums can be connected up to the Gamma 60.

II - The magnetic unit

1. The magnetic tape unit can *read* magnetic tape running either *forwards* or *backwards* at the rate of 1.90m per second, and can write on to magnetic tape at the same speed but *forwards* only. In the latter case, the writing check is performed by an immediate re-reading by the reading head which follows the writing head. This check, as with reading checks, verifies that the check bits read and the catena read agree.

It should be noted that reading or writing directives refer to a block. In whichever direction the tape is read, the central store image of the block read remains unchanged. When reading forwards, the C.A.R. is loaded with the address of the beginning of the C.S. zone where the block is to be stored, and, during transfer, the content of the C.A.R. increases. When reading backwards, however, the C.A.R. is loaded with the end-of-zone address and decreases in the course of the transfer.

2. Reflecting upon their connection with the central unit, the tape units are somewhat different from the other elements. For instance, it was noticed that, when an operation requiring 10 tape units was being carried out, these 10 magnetic tape units never had to operate together ; their role is to place the data required for processing operations at the computer's disposal. Some tape units have to operate continuously (this is often a determining factor in the duration of work) but most of them operate only occasionally. In machines which can carry out only one simultaneous reading and writing operation, it is essential to connect up more than 2 units, and sometimes even 8, 10 or more.

A tape unit needs its local programme only during operation. It was therefore decided to equip the Gamma 60 with fewer local programme circuits than there are tape units ; each local programme circuit is pooled for a certain number of units, and each unit can call several local programme circuits.

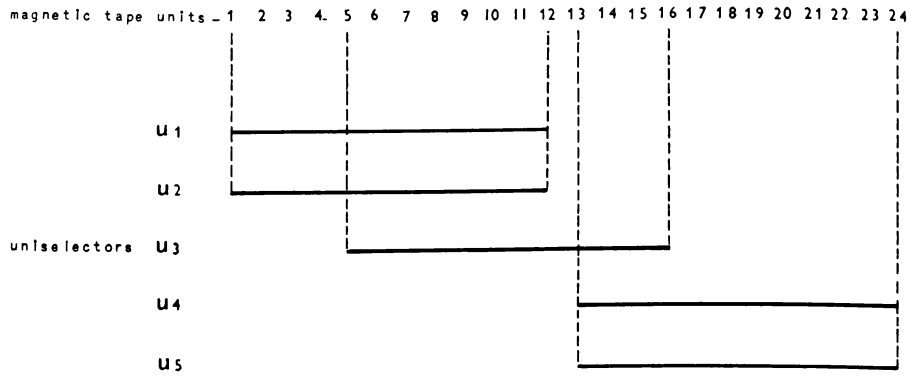
These local programme circuits are called "uniselectors". They are the true elements and the cuts are made on a uniselector number, the tape units being merely pieces of equipment which are temporarily connected up to the uniselectors while they are being used.

The Gamma 60 can be equipped with a maximum of 10 uniselectors, so that, when full use is made of this 10-uniselector capacity, a maximum of 10 tape units may operate simultaneously. Now, magnetic tape unit output is such that, in the event of the above, the Gamma 60 would already be in use for 50% of the time. In addition, the processing of information thus transferred also requires the channels for some cycles....

3. The 10 possible uniselectors are divided into two groups of 5, each group being connected to a maximum of 24 magnetic tape units. 48 magnetic tape units can thus be connected to the Gamma 60. It should be noted incidentally, that even if these 48 magnetic tape units were to have local programmes, they could never operate simultaneously, since the output of 20 of them is enough to saturate the channels for the entire durhtion.

Any one uniselector within each group can be used by 12 different magnetic tape units, and each magnetic tape unit can use 2 or 3 different uniselectors.

Uniselector - magnetic tape unit connections are as show in the following diagram.



Each uniselector group can be partially equipped with magnetic tape units, irrespective of their number. Similarly, a group can consist of only 3 uniselectors, but which must be numbered U1, U2 and U3.

The "small" Gamma 60's have, in fact, only 3 uniselectors and 6 to 8 tape units, but the great majority of Gamma 60's have 5 uniselectors and from 12 to 18 tape units.

4. Magnetic tape programming calls for two types of directives : the preparatory directives, which fix the operating method to be used (forwards or backwards reading, or writing)

and give, in the free variable, the number of the magnetic tape unit for which the uni-selector indicated in the cut is to work ; the others are execution directives which launch the operation block by block (with or without effective transfer of data).

In addition, there are two programming methods which make it possible to use the tape either *discontinuously*, in which case it stops at each block, or *continuously*, which makes it possible to pass stop-start spaces without stopping. These two methods have been programmed once and for all as "standard programmes". It is obvious, however, that if the time it takes to carry out a job depends on the time it takes to read or write a file, then continuous tape movement should be used.

5. Details about the structure and performances of magnetic tape were given in chapter 1, but no mention was made of "mark blocks". These are special blocks filled with 0's on all 8 tracks and over a length of 16 catenae. Special directives (WRM) make it possible to record these blocks without it being necessary to prepare them previously in the central store. When reading takes place, these blocks are identified by the uni-selectors and are not transmitted to the central store, but their detection is reported in the qualitative catena.

They enable a magnetic tape file to be divided into sub-files and blocks containing special data to be located (e.g. sub-file index). These identification blocks are both preceded and followed by a mark block making it possible for a given sub-file to be found on the tape without cluttering up the channels. The whole file can be read, without any transferring, by means of jump directives (JUM), and only the blocks between mark blocks are transferred to the central store for analysis.

6. There is also a "delete" directive which enables *faulty tapes to be used*. The writing check detects the mistake, the faulty section is erased and writing is resumed from there onwards. When reading back, the deleted sections are merely like rather stop-start spaces.

Conclusion.

The place of the Gamma 60 in equipment development

The calculation performances of the so-called "conventional" punched card electro-mechanical machines rapidly proved to be inadequate. This led manufacturers to link them with electronic calculators.

The Gamma 3 was therefore designed in order to work with the tabulator as a straightforward auxiliary calculator. Although much slower than the Gamma 3, the tabulator nevertheless took command, the Gamma 3 operating only when called on.

It was only some time later that the Gamma 3 was equipped with a store - in those days considered to be a high-capacity one - the magnetic drum. This was no longer the mere adding on of a unit, but the creation of a new machine. The store made it possible, indeed, to record not only quantities - data, intermediate results and output results - but also the programme, which could thus itself undergo processing. The tabulator-calculator relationship had changed and the former no longer gave orders to the latter. The tabulator was henceforth confined to the role of a mere input-output element at the service of a calculator which had its own programme.

The Gamma 60 marks yet another decisive step. With its organisation centered around the programme units, it is a data processing machine in the broadest sense of the term. The use of magnetic tape endows it with a practically unlimited storage capacity. The logical organization of its work, which is based on the simultaneous operation of autonomous elements which are specialized in clearly defined functions, gives it power and versatility. The Gamma 60 combines, in a single machine, all the functions of the conventional electro-mechanical installation while, with its operational simultaneity, retaining the same planning facilities. But, in addition to this, over and above the autonomy of the elements are the unity and automation of the machine which blaze the way to the integrated processing of data.

I - Instructions for the central unit

CUTS

Unit	T.O.				C.C.	X	A.F.	
Arithmetic computer Logical unit Transcoder Comparator Magnetic drum Card reader Card punch Printer Magnetic tape Paper tape reader Paper tape punch Control desk	A L T C D C P M T T D	R O R O R A A R A A A E	I G A M U R R I T P P S	T I N P M R P N A R P K	Number of additional conditions : Nothing 1, 2 or 3	Element number	Nothing or R	Nothing or S
Cut on virtual element Liberate virtual element	P L	R I	O B	T E		Address of virtual element image	Nothing or R	Nothing or S
Launch two sequences	S	I	M	U		Address of 2nd branch of programme		
End of sequence	S	T	O	P			Nothing or R	Nothing or S

BRANCHINGS

Operation	T.O.				C.C.	X	A.F.	
Never			0	0		Address of the jump	Nothing or R	Nothing or S
Always			3	1				
Conditional			0 to 2	1 to 4				

ADDRESS INSTRUCTIONS

Operation	T.O.				C.C.	X	A.F.	
Load				L	C.A.R. number 0, 1, 2 or 3	Address or quantity	Nothing or R	Nothing or S
Place				P				
Add				A				
Subtract				S				

CENTRAL DIRECTIVES

Operation	T.O.				C.C.		X				A.F.	
Place instruction number			P	I								
Iteration control			I	C								
Counter progression			C	P								
Transfer in A			T	A								
Place content of A			P	A								
Transfer in B			T	B								
Place content of B			P	B			Address				Nothing, R or M	Nothing or S
Load B (15 right bits)			L	B								
Add in B (15 right bits)			A	B								
Subtract from B (15 right bits)			S	B								
Compare with content of B (15 bits)			C	B								
Substitute quantity in B			S	B			Quantity or address (if S as A.F.)					
Substitute instruction			S	B								
Switch branch to "never"			S	Q								
Switch branch to "always"			S	I								
Reverse switches			F	N								
Fill with zeros				A								
Fill with ones				R			Address				Nothing, R or M	Nothing or S
				Z								
				O								

II - Data processing units

ARITHMETIC UNIT

Operation	T.O.				C.C.		X				A.F.	
							dp	O_1	O_2	R		
Addition		A	D	D	P	D	from 0					
Subtraction		S	U	B	N	or	to 10					
					or	nothing	or					
Multiplication		M	U	L	R		nothing					
Division without remainder		D	I	V	P		from 0					
					or	D	to 10					
Division with remainder		D	I	R	N	or nothing	or nothing					
Transfer		T	R	A	P	D	from 0					
					or	or	to 10					
					N	nothing	or nothing					
Multiply by / Divide by { a power of ten		M	P	T			from 0					
		D	P	T			to 15					
Reverse sign of "P" register		R	S	I								
Take absolute value		A	V	A								
Compare absolute value		C	A	V								
Compare algebraic value		C	A	L								
Round off		R	O	F								

In the columns under headings O_1 and R, the following may be written :
0, 1, 2 or 3 (C.A.R. with progression)
Z, F, S or T (C.A.R. without progression)
P (Register P)
The same applies to O_2 with the exception of P

GENERAL COMPARATOR

Operation	T.O.	C.C.	X	A.F.
One way comparison Two way comparison Delete Transfer from store to store	OWC TWC DEL TSS	1 or 2 1 or 2	Number of catenae (Max : 255)	Nothing or R Nothing or S

TRANSCODER

	T.O.	C.C.	X				A.F.
			dp	01	02	R	
Load the M.I.C.	MIC	LR					
Convert : (phase directives) From BULL cards to processing form From IBM cards to processing form From BULL cards to store form From IBM cards to store form From processing form to BULL cards From paper tape to processing form From processing form to paper tape From processing form to printer From processing form to store form From store form to processing form From store form to store form	BCP IBP IBS IPP PPB PPT PPP PTR PPR PSP SPP SPS	P N or nothing	S or nothing	Part of the M.I.C. used from 1 to 9				
Refer to cliché	RCL	A or nothing	I or 0 –					
End of transcoding	END	S						

THE TRANSLATION CLICHES

Clichés			Function
Positionning	Principal	AW	Alphanumerical word (6 bits per character)
		NW	Numerical word (4 bits per character)
		AN	Arithmetical number (exponent and mantissa - no sign)
		NS	Arithmetical number with sign
		BN	Binary number
	Secondary	AZ	Numerical word, all zeros included
Execution		TO	Transcode one character
		TN	Transfer "N" characters (up to 63 characters)
		IO	Skip input one character
		IN	Skip input "N" characters (up to 63 characters)
		OO	Blank output one character
		ON	Blank output "N" characters (up to 63 characters)
		WO	Write one character
		WN	Write "N" characters (up to 63 characters)
		RO	Write one character, with output address unchanged
		RN	Write "N" characters, with output address unchanged (up to 63 characters)
		RP	Return to programm
		CQ	Write a character in the qualitative catena

LOGICAL COMPUTER

	T.O.			C.C.	X				A.F.
					dp	O_1	O_2	R	
Transfer shift in K		T	S	K	(1)			(2)	
Transfer shift in L		T	S	L				0	
Transfer K in L		T	K	L				1	
Shift left in L		S	H	L				2	
Insert K in L		I	K	L				3	
Fill with zeros in L		F	Z	L				Z	
Fill with ones in L		A	O	L				F	
Addition		S	D	D				S	
Subtraction		M	U	B				T	
Multiplication		D	I	L				or	
Division		F	L	V				L	
Fill L with its complement		I	N	C					
Logical intersection		R	E	T					
Logical reunion		B	L	U					
Binary and logical comparison				C					

from 0 to 23 (number of bits concerned n)

(1) Shift or number of bits or both according to the operations.
(2) L has no meaning for the first two operations.

III - Input output units

CARD READER PUNCH

	T.O.			C.C.	X	A.F.
Select pocket 1 after RS2		S	P	R	1	
Select pocket 2 after RS2		S	P	R	2	
Select pocket 1 after RS5		S	P	P	1	
Select pocket 2 after RS5		S	P	P	2	
Read from hopper 1		R	E	A	M 1	
Read track split after RS5		R	E	A	B 1	
Punch from hopper 1		P	U	N	H 1	
Punch from hopper 2		P	U	N	H 2	
Punch without feeding any card		P	U	N	P	
Track motion :						
- clear the track after reading		C	L	E	B 2	
- clear the track after H2		C	L	E	H 2	
- clear the track after punch		C	L	E	P	
Demand for manipulation - normal (end of file)		D	M	F		
Demand for manipulation - incident		D	M	I		

PAPER TAPE READER

	T.O.				C.C.	X	A.F.
Read		R	E	A		Number of catenae	
Demand for manipulation - normal		D	M	N			
Demand for manipulation - incident		D	M	I			

PAPER TAPE PUNCH

	T.O.				C.C.	X	A.F.
Punch		P	U	N		Number of catenae	
Demand for manipulation - normal		D	M	N			
Demand for manipulation - incident		D	M	I			

PRINTER

	T.O.			C.C.	X	A.F.	
2nd paper skip	S	K	I		Number of spaces Max : 255	Nothing or R	Nothing or S
1st paper skip + printing	P	R	I		id.		
Demand for manipulation - end of file	D	M	F				
Demand for manipulation - incident	D	M	I				

IV - Data storage units

MAGNETIC TAPE

		T.O.			C.C.	X	A.F.	
Preparatory	Write	W	R	I		Magnetic Tape unit if no A.F. Address of tape unit if A.F.	R or nothing	S or nothing
	Read forwards	R	E	F				
	Read backwards	R	E	B				
Execution Directives	Execute reading with transfer	T	R	A				
	Read without transfer (jump)	J	U	M				
	Delete	D	E	L				
	Write mark block	W	R	M				
Service Directives	Spool	S	P	O		Magnetic Tape unit if no A.F. Address of tape unit if A.F.	R or nothing	S or nothing
	Spool and demand for manipulation - normal	S	P	N				
	Demand for manipulation - incident	M	A	I				
	Demand for manipulation - normal	M	A	N				

MAGNETIC DRUM

	T.O.			C.C.	X	A.F.	
Designation of the address of the origin	D	A	O		Address on the drum	R or nothing	S or nothing
Transfer central store to drum	T	C	D		Number of catenae		
Transfer drum to central store	T	D	C				

TABLE OF CONTENTS

INTRODUCTION 3

Chapter I - DATA IN THE GAMMA 60 7

I - Data and machines 8
II - The store function 9
III - The stores in the Gamma 60 10
IV - The infernal language of the Gamma 60 15

Chapter II - GENERAL DESIGN OF THE GAMMA 60 19

I - The governing principle of the Gamma 60 20
II - Brief description of the structure of the Gamma 60 20
III - The elements . their connection with the central unit 22
IV - Co-ordination of the operation of the elements 23
V - Checking in the Gamma 60 25
VI - Conclusion 26

Chapter III - ELEMENTARI INSTRUCTIONS 27

I - The different types of instructions : code B 28
II - Structure of an elementary instruction in code B 28
III - Cut instructions 30
IV - Branchings 31
V - Address instructions 32
VI - Directives 32

Chapter IV - DATA PROCESSING UNITS 33

I - The arithmetic unit 35
II - The general comparator 36
III - The transcoder 37
IV - The logical unit 39

Chapter V - INPUT-OUTPUT UNITS 41

I - The card reader-punch 42
II - The card reader 43
III - The paper tape reader 43
IV - The paper tape punch 45
V - The printer 45
VI - The Gamma 60 control desk 47

Chapter VI - DATA STORAGE UNITS 51

I - The magnetic drum 53
II - The magnetic unit 53

Conclusion 57

The Gamma 60 code B table of instructions 59